

**TRƯỜNG ĐẠI HỌC NGUYỄN TRÃI**  
**KHOA CÔNG NGHỆ THÔNG TIN & CHUYỂN ĐỔI SỐ**



**ĐẠI HỌC NGUYỄN TRÃI**  
**TRUNG THỰC**

**BÁO CÁO**  
**KẾT THÚC HỌC PHẦN**

**ĐỀ TÀI**  
**BẢO MẬT ỨNG DỤNG VÀ PHẦN MỀM**

**Học phần**                      **An toàn và bảo mật thông tin**  
**Lớp:**                              **K23CNT2**  
**GVHD:**                        **ThS. GVC Nguyễn Tài Tuyên**  
**Nhóm sinh viên:**   **Nhóm 9**

**Hà Nội, Tháng 6 năm 2026**

**TRƯỜNG ĐẠI HỌC NGUYỄN TRÃI**  
**KHOA CÔNG NGHỆ THÔNG TIN & CHUYỂN ĐỔI SỐ**



**ĐẠI HỌC NGUYỄN TRÃI**  
**TRUNG THỰC**

# **BÁO CÁO**

## **KẾT THÚC HỌC PHẦN**

### **ĐỀ TÀI**

#### **BẢO MẬT ỨNG DỤNG VÀ PHẦN MỀM**

**Danh sách nhóm sinh viên:**

| <b>STT</b> | <b>Họ và tên</b>    | <b>Mã SV</b> | <b>Nội Dung</b>                                   | <b>Ký tên</b> |
|------------|---------------------|--------------|---------------------------------------------------|---------------|
| <b>1</b>   | Lục Văn Sơn         | 2310900087   | Cơ sở lý thuyết và thực hành<br>OWASP ZAP, DVWA   |               |
| <b>2</b>   | Nguyễn Công<br>Tùng | 2310900051   | Cơ sở lý thuyết và thực hành<br>SQL injection     |               |
| <b>3</b>   | Đinh Tiến<br>Triển  | 2310900107   | Cơ sở lý thuyết và thực hành<br>Trivy, Dockerfile |               |
| <b>4</b>   | Tạ Văn Tân          | 2310900095   | Cơ sở lý thuyết chương 1                          |               |

**Nhóm trưởng: Lục Văn Sơn**

**Hà Nội, Tháng 6 năm 2026**

## LỜI CAM ĐOAN

Chúng em xin cam đoan tiểu luận môn học An toàn và bảo mật hệ thống thông tin với đề tài "Tìm hiểu cấu trúc bảo mật ứng dụng và thực hành các kỹ thuật kiểm thử - DevSecOps" là kết quả do chính nhóm chúng em thực hiện dưới sự hướng dẫn của giảng viên TS. GVC Nguyễn Tài Tuyên. Các số liệu, kết quả thực nghiệm nêu trong tiểu luận là trung thực; mọi tài liệu tham khảo, hình ảnh và đoạn mã của tác giả khác đều được trích dẫn đầy đủ. Chúng em xin chịu trách nhiệm hoàn toàn về nội dung của bản báo cáo này.

Nhóm Sinh viên thực hiện  
(Ký và ghi rõ họ tên)

# MỤC LỤC

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| CHƯƠNG 1. CƠ SỞ LÝ THUYẾT .....                                              | 3  |
| 1.1. Vòng đời phát triển phần mềm an toàn (Secure SDLC).....                 | 3  |
| 1.1.1. <i>Khái niệm Secure SDLC</i> .....                                    | 3  |
| 1.1.2. <i>Các giai đoạn trong Secure SDLC</i> .....                          | 3  |
| 1.2. Các rủi ro bảo mật ứng dụng web (OWASP Top 10 - 2021) .....             | 4  |
| 1.2.1. <i>Giới thiệu về OWASP và OWASP Top 10</i> .....                      | 4  |
| 1.2.2. <i>Danh sách 10 rủi ro hàng đầu theo OWASP Top 10 – 2021</i> .....    | 5  |
| 1.2.3. <i>Ứng dụng của OWASP Top 10 trong thực tiễn và Secure SDLC</i> ....  | 7  |
| 1.3. Các phương pháp kiểm thử bảo mật ứng dụng.....                          | 8  |
| 1.3.1. <i>Khái niệm và mục tiêu của kiểm thử bảo mật ứng dụng</i> .....      | 8  |
| 1.3.2. <i>Chiến lược kết hợp và vị trí trong Secure SDLC</i> .....           | 10 |
| 1.4. Bảo mật hạ tầng và môi trường hiện đại .....                            | 11 |
| 1.4.1. <i>Khái niệm và bối cảnh</i> .....                                    | 11 |
| 1.4.2. <i>Các thành phần chính và rủi ro bảo mật đặc thù</i> .....           | 11 |
| 1.4.3. <i>Các nguyên tắc và biện pháp bảo mật cho hạ tầng hiện đại</i> ..... | 13 |
| 1.4.4. <i>Mối quan hệ với Secure SDLC và kiểm thử bảo mật</i> .....          | 14 |
| CHƯƠNG 2. NỘI DUNG NGHIÊN CỨU.....                                           | 15 |
| 2.1. OWASP ZAP (Zed Attack Proxy).....                                       | 15 |
| 2.1.1. <i>Khái niệm và mục đích</i> .....                                    | 15 |
| 2.1.2. <i>Kiến trúc và nguyên lý hoạt động</i> .....                         | 15 |
| 2.1.3. <i>Các tính năng chính</i> .....                                      | 15 |
| 2.1.4. <i>Vai trò trong bối cảnh nghiên cứu</i> .....                        | 16 |
| 2.2. Burp Suite (Community Edition).....                                     | 16 |
| 2.2.1. <i>Khái niệm và phân khúc</i> .....                                   | 16 |
| 2.2.2. <i>Cơ chế hoạt động và các thành phần chính</i> .....                 | 16 |
| 2.2.3. <i>Thế mạnh so với OWASP ZAP (trong bản miễn phí)</i> .....           | 17 |
| 2.2.4. <i>Vai trò trong bối cảnh nghiên cứu</i> .....                        | 17 |
| 2.3. Môi trường mục tiêu: DVWA (Damn Vulnerable Web App).....                | 17 |
| 2.3.1. <i>Khái niệm và mục đích</i> .....                                    | 17 |
| 2.3.2. <i>Các tính năng chính phục vụ học tập và nghiên cứu</i> .....        | 17 |

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| 2.3.3. <i>Vai trò trong bối cảnh nghiên cứu</i> .....                        | 18 |
| 2.4. Công cụ kiểm tra Container và CI/CD.....                                | 18 |
| 2.4.1. <i>Bối cảnh: Bảo mật trong môi trường hiện đại</i> .....              | 18 |
| 2.4.2. <i>Trivy: Công cụ quét bảo mật đa năng cho Container và IaC</i> ..... | 18 |
| 2.4.3. <i>Các tính năng chính và nguyên lý</i> .....                         | 18 |
| 2.5. SQL Injection – Cơ sở lý thuyết và phương pháp phòng chống .....        | 19 |
| 2.5.1. <i>Khái niệm</i> .....                                                | 19 |
| 2.5.2. <i>Các dạng tấn công SQL Injection phổ biến</i> .....                 | 19 |
| 2.5.3. <i>Nguyên tắc phòng chống SQL Injection</i> .....                     | 19 |
| 2.6. Bảo mật Container với Dockerfile an toàn và Trivy.....                  | 20 |
| 2.6.1. <i>Bối cảnh và rủi ro từ container</i> .....                          | 20 |
| 2.6.2. <i>Nguyên tắc viết Dockerfile an toàn</i> .....                       | 20 |
| 2.6.3. <i>Trivy – Công cụ quét bảo mật đa năng</i> .....                     | 20 |
| 2.7. Nền tảng điều phối container cho môi trường hiện đại.....               | 21 |
| 2.7.1. <i>Khái niệm và vai trò</i> .....                                     | 21 |
| 2.7.2. <i>Kiến trúc tổng quan</i> .....                                      | 21 |
| 2.7.3. <i>Các rủi ro bảo mật đặc thù trong Kubernetes</i> .....              | 22 |
| 2.8. Kiến trúc và bảo mật trong môi trường hiện đại.....                     | 23 |
| 2.8.1. <i>Khái niệm Cloud Native</i> .....                                   | 23 |
| 2.8.2. <i>Các đặc trưng chính của Cloud Native</i> .....                     | 23 |
| CHƯƠNG 3. THỰC NGHIỆM / GIẢI PHÁP ĐỀ XUẤT .....                              | 25 |
| 3.1. <i>Môi trường và công cụ thử nghiệm</i> .....                           | 25 |
| 3.2. <i>Các bước thực hiện</i> .....                                         | 25 |
| 3.3. <i>Kết quả và nhận xét</i> .....                                        | 26 |
| 3.4. <i>Đề xuất biện pháp phòng chống / khắc phục</i> .....                  | 30 |
| KẾT LUẬN.....                                                                | 31 |
| TÀI LIỆU THAM KHẢO.....                                                      | 32 |
| PHỤ LỤC.....                                                                 | 33 |
| PHỤ LỤC A: ĐÁNH GIÁ VÀ XỬ LÝ LỖ HỔNG ỨNG DỤNG (WEB VULNERABILITY).....       | 33 |
| PHỤ LỤC B: CHIẾN LƯỢC TỐI ƯU HÓA VÀ BẢO MẬT CONTAINER (DOCKER) .....         | 33 |
| PHỤ LỤC C: BẢO MẬT HẠ TẦNG ĐIỀU PHỐI (KUBERNETES).....                       | 34 |



## DANH MỤC TỪ VIẾT TẮT

| STT | Từ viết tắt | Cụm từ đầy đủ                                                                              |
|-----|-------------|--------------------------------------------------------------------------------------------|
| 1   | AST         | Application Security Testing / Kiểm thử bảo mật ứng dụng.                                  |
| 2   | ASVS        | Application Security Verification Standard / Tiêu chuẩn kiểm chứng an ninh ứng dụng.       |
| 3   | BSIMM       | Building Security In Maturity Model.                                                       |
| 4   | CI-Fuzzing  | Continuous Integration Fuzzing.                                                            |
| 5   | CNAPP       | Cloud Native App Protection.                                                               |
| 6   | CSPM        | Cloud Security Posture Management.                                                         |
| 7   | CSRF        | Cross-Site Request Forgery.                                                                |
| 8   | CVE         | Common Vulnerabilities and Exposures.                                                      |
| 9   | CVSS        | Common Vulnerability Scoring System.                                                       |
| 10  | CWPP        | Cloud Workload Protection Platform.                                                        |
| 11  | DAST        | Dynamic Application Security Testing / Phân tích động.                                     |
| 12  | DFD         | Data Flow Diagram.                                                                         |
| 13  | DREAD       | Damage, Reproducibility, Exploitability, Affected users, Discoverability.                  |
| 14  | DVWA        | Damn Vulnerable Web Application.                                                           |
| 15  | IAST        | Interactive Application Security Testing.                                                  |
| 16  | IaaS        | Cơ sở hạ tầng dưới dạng dịch vụ.                                                           |
| 17  | IAM         | Identity and Access Management.                                                            |
| 18  | IaC         | Infrastructure as Code / Hạ tầng được định nghĩa bằng mã.                                  |
| 19  | IDOR        | Insecure Direct Object Reference.                                                          |
| 20  | JWT         | JSON Web Token.                                                                            |
| 21  | K8s         | Kubernetes.                                                                                |
| 22  | mTLS        | mutual TLS.                                                                                |
| 23  | OS          | Hệ điều hành.                                                                              |
| 24  | OWASP       | Open Worldwide Application Security Project / Open Web Application Security Project.       |
| 25  | PASTA       | Process for Attack Simulation and Threat Analysis.                                         |
| 26  | PaaS        | Nền tảng (dưới dạng dịch vụ).                                                              |
| 27  | PSA         | Pod Security Admission.                                                                    |
| 28  | PSS         | Pod Security Standards.                                                                    |
| 29  | PT          | Penetration Testing / Kiểm thử thâm nhập.                                                  |
| 30  | PTES        | Penetration Testing Execution Standard.                                                    |
| 31  | RASP        | Runtime Application Self-Protection / Runtime App Self-Protection / Bảo vệ thời gian thực. |
| 32  | RBAC        | Role-Based Access Control / Phân quyền kiểm soát truy cập.                                 |
| 33  | SaaS        | Phần mềm (dưới dạng dịch vụ).                                                              |
| 34  | SAMM        | Software Assurance Maturity Model.                                                         |
| 35  | SAST        | Static Application Security Testing / Phân tích mã nguồn tĩnh.                             |
| 36  | SBOM        | Software Bill of Materials.                                                                |
| 37  | SCA         | Software Composition Analysis.                                                             |

|    |        |                                                                                                                        |
|----|--------|------------------------------------------------------------------------------------------------------------------------|
| 38 | SDLC   | Secure Software Development Life Cycle / Secure Software Development Lifecycle / Vòng đời phát triển phần mềm an toàn. |
| 39 | SQLi   | SQL Injection.                                                                                                         |
| 40 | SSDF   | Secure Software Development Framework.                                                                                 |
| 41 | SSRF   | Server-Side Request Forgery / Giả mạo yêu cầu phía máy chủ.                                                            |
| 42 | STRIDE | Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation of privilege.                                 |
| 43 | UAT    | Kiểm thử chấp nhận.                                                                                                    |
| 44 | VA     | Vulnerability Assessment / Đánh giá lỗ hổng.                                                                           |
| 45 | WSTG   | Web Security Testing Guide.                                                                                            |
| 46 | XSS    | Cross-Site Scripting.                                                                                                  |
| 47 | ZAP    | Zed Attack Proxy.                                                                                                      |



# MỞ ĐẦU

## 1. Lý do chọn đề tài

Trong kỷ nguyên chuyển đổi số hiện nay, các ứng dụng web và phần mềm đóng vai trò như hệ thống thần kinh trung ương của hầu hết các tổ chức và doanh nghiệp. Sự gia tăng theo cấp số nhân của các dịch vụ trực tuyến kéo theo sự gia tăng của các mối đe dọa an ninh mạng. Hàng loạt các cuộc tấn công khai thác lỗ hổng ứng dụng (như SQL Injection, Broken Access Control, hay các cuộc tấn công vào chuỗi cung ứng phần mềm) đã gây ra những thiệt hại nặng nề về tài chính và uy tín.

Thực tế này đặt ra yêu cầu cấp thiết về việc tích hợp bảo mật vào mọi giai đoạn của quá trình phát triển phần mềm, thay vì chỉ kiểm tra ở khâu cuối cùng. Mô hình Secure SDLC (Vòng đời phát triển phần mềm an toàn) và triết lý DevSecOps ra đời nhằm giải quyết vấn đề này. Nhận thức được tầm quan trọng đó, nhóm chúng em quyết định chọn đề tài "**Tìm hiểu cấu trúc bảo mật ứng dụng và thực hành các kỹ thuật kiểm thử - DevSecOps**" làm tiểu luận môn học. Đề tài không chỉ đi sâu vào lý thuyết (OWASP Top 10, SAST, DAST) mà còn áp dụng thực tiễn qua các công cụ chuyên chuẩn công nghiệp như OWASP ZAP, Burp Suite, Trivy và tích hợp CI/CD.

## 2. Mục tiêu nghiên cứu

- **Mục tiêu 1:** Hệ thống hóa cơ sở lý thuyết về bảo mật ứng dụng, vòng đời Secure SDLC, các phương pháp kiểm thử (SAST, DAST, IAST) và danh sách lỗ hổng OWASP Top 10.
- **Mục tiêu 2:** Nghiên cứu các phương pháp bảo mật hạ tầng hiện đại bao gồm bảo mật API, Container (Docker) và môi trường phân tán Kubernetes.
- **Mục tiêu 3:** Triển khai thực nghiệm (Lab) các kỹ thuật đánh giá lỗ hổng và tích hợp bảo mật tự động thông qua nền tảng GitHub Actions (DevSecOps).

## 3. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Các lỗ hổng bảo mật ứng dụng web phổ biến, các công cụ kiểm thử bảo mật mã nguồn mở, kiến trúc bảo mật Container.
- **Phạm vi nghiên cứu:** Tập trung vào các lỗ hổng theo chuẩn OWASP Top 10 (2021). Các bài thực hành được thực hiện trên môi trường máy ảo, ứng dụng chủ đích chứa lỗ hổng (DVWA) và hệ thống CI/CD mô phỏng.

## 4. Phương pháp nghiên cứu

- **Phương pháp tổng hợp tài liệu:** Thu thập thông tin từ các slide bài giảng, chuẩn bảo mật OWASP, tài liệu của NIST và các báo cáo an toàn thông tin quốc tế.
- **Phương pháp thực nghiệm (Lab-based):** Sử dụng các công cụ (OWASP ZAP, Burp Suite, Trivy, Semgrep) để tấn công giả lập và phòng thủ trên môi trường được kiểm soát an toàn.

## 5. Bố cục của tiểu luận

Ngoài phần Mở đầu, Kết luận và Tài liệu tham khảo, nội dung tiểu luận gồm ... chương:

- **Chương 1:** Cơ sở lý thuyết.
- **Chương 2:** Nội dung nghiên cứu (Các công cụ và nền tảng).
- **Chương 3:** Thực nghiệm và Giải pháp đề xuất.

# CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

## 1.1. Vòng đời phát triển phần mềm an toàn (Secure SDLC)

### 1.1.1. Khái niệm Secure SDLC

Secure Software Development Life Cycle (Secure SDLC) hay Vòng đời phát triển phần mềm an toàn, được hiểu là một quy trình phát triển phần mềm có tích hợp các biện pháp kiểm soát an ninh thông tin vào tất cả các giai đoạn của quy trình phát triển truyền thống. Nó không phải là một phương pháp luận thay thế, mà là một lớp bảo vệ được phủ lên trên quy trình SDLC hiện có, bổ sung các hoạt động và công cụ bảo mật cụ thể vào từng bước [1], [14].

Mục tiêu cốt lõi của Secure SDLC là chuyển đổi bảo mật từ một khâu kiểm tra cuối cùng thành một yếu tố được xem xét xuyên suốt, từ khâu lên ý tưởng đến khi vận hành phần mềm. Cách tiếp cận này được gọi là "Shift Left" – tức là đưa bảo mật lên sớm ngay từ giai đoạn đầu của vòng đời, thay vì đợi đến khi hoàn thiện mới kiểm tra. Điều này có ý nghĩa quan trọng trong bối cảnh các chuỗi công cụ phần mềm ngày càng phức tạp, với nhiều thành phần nguồn mở, API bên ngoài và cơ sở hạ tầng đám mây[9].

Việc áp dụng Secure SDLC mang lại nhiều lợi ích thiết thực. Về mặt kỹ thuật, nó giúp xác định và giảm thiểu các lỗ hổng bảo mật ngay từ khi chúng mới hình thành, qua đó giảm đáng kể rủi ro cho sản phẩm cuối cùng. Về mặt kinh tế, việc phát hiện và sửa lỗi bảo mật ở giai đoạn sớm sẽ có chi phí thấp hơn nhiều lần so với khi phần mềm đã được đưa vào sản xuất. Về mặt tổ chức, cách tiếp cận này giúp xây dựng một thể trận an ninh vững chắc, bảo vệ doanh nghiệp trước các mối đe dọa ngày càng tinh vi và giúp đáp ứng các yêu cầu tuân thủ một cách hiệu quả.[3]

### 1.1.2. Các giai đoạn trong Secure SDLC

Mặc dù có thể được điều chỉnh linh hoạt tùy theo mô hình phát triển (ví dụ Waterfall, Agile hay DevOps) và bối cảnh cụ thể của mỗi tổ chức, Secure SDLC thường được cấu trúc dựa trên năm giai đoạn chính[14]. Trong mỗi giai đoạn, các hoạt động bảo mật đặc thù sẽ được thực thi, tạo thành một quy trình kiểm soát toàn diện.

- **Giai đoạn 1: Yêu cầu (Requirements):** Ở giai đoạn này, bên cạnh các yêu cầu chức năng và nghiệp vụ, các yêu cầu về an toàn thông tin cũng phải được xác định rõ ràng, dựa trên các quy định pháp lý, các tiêu chuẩn quốc tế và chính sách nội bộ của tổ chức. Các yêu cầu này có thể bao gồm việc tuân thủ các chuẩn mực nhất định (ví dụ: GDPR, HIPAA) hoặc các cơ chế bảo mật kỹ thuật bắt buộc như mã hóa dữ liệu nhạy cảm, phân quyền kiểm soát truy cập RBAC, v.v.. Việc xác định sớm các yêu cầu này là nền tảng để tính toán chi phí và lập kế hoạch cho các giai đoạn tiếp theo.
- **Giai đoạn 2: Thiết kế (Design):** Dựa trên các yêu cầu đã được phê duyệt, kiến trúc tổng thể của sản phẩm được xây dựng. Nhiệm vụ chính của giai đoạn này là đảm bảo các quyết định về thiết kế không tạo ra lỗ hổng bảo mật. Các chuyên gia an ninh sẽ tiến hành mô hình hóa mối đe dọa (Threat Modeling) để xác định các tình huống tấn công tiềm ẩn, đánh giá rủi ro và

lên kế hoạch cho các biện pháp bảo vệ tương ứng. Bất kỳ sự thay đổi nào trong thiết kế đều cần được đánh giá lại tác động đến mô hình bảo mật.

- **Giai đoạn 3: Phát triển (Development/Implementation):** Đây là giai đoạn viết mã nguồn dựa trên các yêu cầu và thiết kế đã được phê duyệt. Bảo mật ở giai đoạn này được đảm bảo thông qua việc áp dụng các quy tắc lập trình an toàn (Secure Coding), cùng với việc sử dụng các công cụ phân tích tĩnh để phát hiện lỗi hổng ngay trong mã nguồn. Ba công cụ chính thường được sử dụng bao gồm:
  - **SAST (Static Application Security Testing):** Phân tích mã nguồn tĩnh để tìm ra các lỗi bảo mật phổ biến.
  - **SCA/OSA (Software Composition Analysis):** Phân tích các thư viện và thành phần mã nguồn mở mà dự án sử dụng để phát hiện các lỗ hổng đã biết.
  - **CI-Fuzzing (Continuous Integration Fuzzing):** Kỹ thuật kiểm thử "mò" được thực hiện trên các hàm cụ thể trong quá trình tích hợp liên tục để tìm ra các lỗi bất ngờ.
  - **Giai đoạn 4: Kiểm thử (Testing):** Sau khi mã nguồn hoàn thiện, giai đoạn kiểm thử tập trung vào việc đánh giá bảo mật của toàn bộ hệ thống trong một môi trường vận hành thử nghiệm. Các hoạt động chính bao gồm kiểm thử xâm nhập (Penetration Testing) để mô phỏng các cuộc tấn công thực tế, và phân tích động (DAST - Dynamic Application Security Testing) để kiểm tra ứng dụng trong khi nó đang chạy. Mục đích là phát hiện các lỗ hổng xuất phát từ sự tương tác giữa các thành phần hoặc các vấn đề cấu hình mà phân tích tĩnh chưa thể tìm ra.
- **Giai đoạn 5: Triển khai và Bảo trì (Deployment & Maintenance):** Đây là giai đoạn cuối cùng, nơi phần mềm chính thức được đưa vào môi trường sản xuất. Các hoạt động bảo mật vẫn tiếp tục diễn ra để đảm bảo "thế trận an ninh" không bị suy giảm theo thời gian. Các hoạt động quan trọng bao gồm: quản lý lỗ hổng sau khi phát hành, thiết lập các quy trình ứng phó sự cố, thực hiện đánh giá bảo mật định kỳ, và quan trọng là duy trì một quy trình cập nhật bản vá an toàn để bảo vệ phần mềm trước các mối đe dọa mới.

## 1.2. Các rủi ro bảo mật ứng dụng web (OWASP Top 10 - 2021)

### 1.2.1. Giới thiệu về OWASP và OWASP Top 10

OWASP (Open Worldwide Application Security Project) là một tổ chức phi lợi nhuận hoạt động trên phạm vi toàn cầu, chuyên cải thiện an ninh cho các ứng dụng web, ứng dụng di động và dịch vụ API. OWASP vận hành dựa trên mô hình cộng đồng mở, nơi hàng nghìn chuyên gia bảo mật, lập trình viên và nhà nghiên cứu cùng đóng góp để xây dựng các tiêu chuẩn, công cụ và tài liệu hướng dẫn miễn phí. Các dự án nổi bật nhất của OWASP bao gồm OWASP ZAP (công cụ kiểm thử xâm nhập mã nguồn mở), OWASP ASVS (tiêu chuẩn kiểm chứng an ninh ứng dụng) và đặc biệt là OWASP Top

10, tài liệu được coi là chuẩn mực tham chiếu về các rủi ro bảo mật nghiêm trọng nhất [2], [14].

OWASP Top 10 là một tài liệu tiêu chuẩn về nhận thức, được xây dựng từ dữ liệu thực tế trên hàng triệu ứng dụng được kiểm tra toàn cầu. Tài liệu này đại diện cho sự đồng thuận rộng rãi của cộng đồng về những rủi ro bảo mật nguy hiểm nhất mà các ứng dụng web phải đối mặt. Điểm đặc biệt của OWASP Top 10 là nó không phải một danh sách tĩnh mà liên tục thay đổi qua từng phiên bản để phản ánh chính xác bối cảnh an ninh mạng hiện tại. Phiên bản 2021, được phát hành sau 4 năm kể từ phiên bản 2017, đã có những sự điều chỉnh quan trọng về thứ tự, sáp nhập và bổ sung một số hạng mục mới, phản ánh các mối đe dọa và xu hướng tấn công trong giai đoạn đó.

Vai trò và ý nghĩa của OWASP Top 10 trong bảo mật ứng dụng web là vô cùng quan trọng. Nó được xem là "kim chỉ nam" cung cấp một điểm khởi đầu có hệ thống để nâng cao bảo mật cho ứng dụng web. Đối với các tổ chức và nhà phát triển, việc áp dụng OWASP Top 10 giúp:

- Nâng cao nhận thức bảo mật cho toàn bộ đội ngũ phát triển.
- Xác định và ưu tiên xử lý các lỗ hổng phổ biến và nguy hiểm nhất.
- Tích hợp bảo mật vào quy trình phát triển (Secure SDLC).
- Tạo ra một thước đo chung để đánh giá và so sánh mức độ an toàn giữa các ứng dụng.
- Xây dựng một chương trình an ninh ứng dụng (AppSec Program) hiệu quả.

Không chỉ dừng lại ở việc liệt kê, OWASP Top 10 còn cung cấp các hướng dẫn cụ thể về cách phòng tránh, khắc phục và giảm thiểu từng rủi ro, khiến nó trở thành một công cụ thực tiễn không thể thiếu trong bộ kỹ năng của bất kỳ ai liên quan đến an toàn thông tin ứng dụng web.

### ***1.2.2. Danh sách 10 rủi ro hàng đầu theo OWASP Top 10 – 2021***

Danh sách OWASP Top 10:2021 bao gồm 10 hạng mục rủi ro, được sắp xếp theo mức độ nghiêm trọng dựa trên tần suất xuất hiện, mức độ ảnh hưởng và khả năng khai thác [2]. Phân tích chi tiết từng rủi ro như sau:

1. A01:2021 - Broken Access Control (Kiểm soát truy cập bị hỏng): Đây là rủi ro nghiêm trọng nhất và đã vươn lên vị trí số 1 trong phiên bản 2021, thay thế cho Injection ở vị trí cũ. Lỗ hổng này xảy ra khi các cơ chế kiểm soát quyền truy cập của ứng dụng không được thực thi đúng cách. Hậu quả là kẻ tấn công có thể khai thác để xem, sửa, xóa dữ liệu trái phép, thực hiện các chức năng vượt quá thẩm quyền hoặc thậm chí kiểm soát toàn bộ tài khoản quản trị viên. Có tới 94% ứng dụng được kiểm tra cho thấy có dạng lỗ hổng này. Các biện pháp giảm thiểu bao gồm áp dụng nguyên tắc tối thiểu đặc quyền (Least Privilege), từ chối mọi truy cập theo mặc định, và thực hiện kiểm soát truy cập dựa trên vai trò (RBAC) [8], [14].
2. A02:2021 - Cryptographic Failures (Thất bại mật mã học): Trước đây được gọi là "Sensitive Data Exposure" (Phơi lộ dữ liệu nhạy cảm), rủi ro này

chuyển từ vị trí thứ 3 lên thứ 2 trong năm 2021. Nó đề cập đến các lỗ hổng xuất phát từ việc sử dụng mật mã không đúng cách hoặc không sử dụng mật mã, dẫn đến việc lộ lọt dữ liệu nhạy cảm. Các nguyên nhân điển hình gồm sử dụng các thuật toán mã hóa yếu, quản lý khóa kém, hoặc không mã hóa dữ liệu khi truyền tải (in-transit) cũng như khi lưu trữ (at-rest). Việc giảm thiểu đòi hỏi phải mã hóa toàn bộ dữ liệu nhạy cảm, sử dụng các giao thức và thuật toán mạnh (như TLS 1.3, AES-256), và không lưu trữ dữ liệu nhạy cảm trừ khi thực sự cần thiết [8], [14].

3. A03:2021 - Injection (Tiêm mã): Là "vua" của các lỗ hổng trong các phiên bản trước (2013, 2017), Injection đã tụt xuống vị trí thứ 3 trong năm 2021. Lỗ hổng này xảy ra khi dữ liệu không đáng tin cậy được gửi tới bộ phân tích cú pháp (interpreter) như một phần của câu lệnh hoặc truy vấn, chẳng hạn như SQL, NoSQL, hoặc LDAP. Kẻ tấn công có thể lợi dụng điều này để thực thi các câu lệnh độc hại, truy cập hoặc phá hủy dữ liệu trái phép. Điểm đáng chú ý trong phiên bản 2021 là lỗ hổng Cross-Site Scripting (XSS) đã được sáp nhập vào danh mục Injection này. Mặc dù đã giảm thứ hạng, Injection vẫn là một mối đe dọa phổ biến, và giải pháp phòng tránh hiệu quả nhất là sử dụng truy vấn có tham số (parameterized queries) và xác thực đầu vào (input validation) [8], [14].
4. A04:2021 - Insecure Design (Thiết kế không an toàn): Đây là một hạng mục hoàn toàn mới được bổ sung trong ấn bản năm 2021, nhấn mạnh vào các rủi ro phát sinh từ các lỗ hổng trong khâu thiết kế của ứng dụng. Khác với các lỗi ở tầng mã nguồn (coding), lỗi thiết kế mang tính hệ thống, sâu xa và thường rất tốn kém để sửa chữa sau khi ứng dụng đã được triển khai. Các vấn đề như phân tách dữ liệu giữa các khách hàng (multi-tenancy) kém, hoặc logic nghiệp vụ sai lệch dẫn đến lỗ hổng là những ví dụ điển hình. Việc phòng tránh yêu cầu phải tích hợp bảo mật từ giai đoạn thiết kế, thực hiện mô hình hóa mối đe dọa (Threat Modeling) và các phiên đánh giá thiết kế an ninh [8], [14].
5. A05:2021 - Security Misconfiguration (Cấu hình bảo mật sai): Rủi ro này xảy ra khi các thiết lập bảo mật của ứng dụng, máy chủ web, cơ sở dữ liệu hoặc các dịch vụ khác bị cấu hình không chính xác hoặc không đầy đủ. Các ví dụ bao gồm: giữ lại các tài khoản mặc định và mật khẩu không đổi, bật tính năng liệt kê thư mục (directory listing), để các cổng (ports) không cần thiết ở trạng thái mở, hoặc sử dụng các header bảo mật HTTP (như CSP, HSTS) không phù hợp. Đây là một trong những lỗ hổng phổ biến nhất và thường dễ bị khai thác nhất [8].
6. A06:2021 - Vulnerable and Outdated Components (Thành phần dễ bị tổn thương và lỗi thời): Rủi ro này nhấn mạnh thực tế rằng hầu hết các ứng dụng hiện đại đều được xây dựng dựa trên nhiều thư viện, framework và thành phần mã nguồn mở. Khi các thành phần này có chứa lỗ hổng bảo mật đã được công bố hoặc đã quá cũ, không được hỗ trợ, chúng trở thành điểm xâm nhập dễ dàng cho kẻ tấn công. Việc giảm thiểu đòi hỏi phải có một quy trình

quản lý thành phần hiệu quả, bao gồm việc liên tục cập nhật và vá lỗi cho tất cả các thư viện và framework được sử dụng[8].

7. A07:2021 - Identification and Authentication Failures (Thất bại nhận dạng và xác thực): Hạng mục này (trước đây có tên là "Broken Authentication") đề cập đến các lỗ hổng liên quan đến việc xác thực danh tính của người dùng. Các vấn đề điển hình bao gồm: cho phép sử dụng mật khẩu yếu, cơ chế quản lý phiên (session) không an toàn (ví dụ: session ID có thể dễ dàng dự đoán), hoặc không có cơ chế bảo vệ chống lại các cuộc tấn công dò mật khẩu (credential stuffing, brute-force). Hậu quả là kẻ tấn công có thể chiếm quyền điều khiển tài khoản hợp pháp của người dùng[8].
8. A08:2021 - Software and Data Integrity Failures (Thất bại toàn vẹn phần mềm và dữ liệu): Đây là một hạng mục mới khác trong phiên bản 2021, phản ánh các rủi ro đến từ việc giả định rằng các thành phần phần mềm là đáng tin cậy mà không có sự xác minh. Các lỗ hổng điển hình bao gồm: tải xuống và sử dụng các thư viện từ nguồn không an toàn (không qua HTTPS, không kiểm tra chữ ký), hoặc các lỗ hổng "Insecure Deserialization" (giải tuần tự hóa không an toàn) từ các phiên bản cũ. Việc sử dụng pipeline CI/CD không đảm bảo tính toàn vẹn của mã nguồn cũng nằm trong phạm vi của rủi ro này[8].
9. A09:2021 - Security Logging and Monitoring Failures (Thất bại ghi nhật ký và giám sát bảo mật): Rủi ro này cho thấy tầm quan trọng của khả năng phát hiện sự cố. Khi một hệ thống không ghi lại đầy đủ các sự kiện bảo mật (ví dụ: đăng nhập thất bại, thay đổi quyền, truy cập trái phép) hoặc không có cơ chế giám sát, cảnh báo hiệu quả, kẻ tấn công có thể tự do hoạt động mà không bị phát hiện trong một thời gian dài. Điều này dẫn đến việc các vụ vi phạm dữ liệu chỉ được phát hiện từ bên thứ ba (ví dụ: cơ quan thực thi pháp luật) hoặc quá muộn để có thể ngăn chặn thiệt hại[8].
10. A10:2021 - Server-Side Request Forgery (SSRF - Giả mạo yêu cầu phía máy chủ): Đây là hạng mục cuối cùng và cũng là một hạng mục mới được bổ sung trong phiên bản 2021. Lỗ hổng SSRF xảy ra khi ứng dụng web lấy dữ liệu từ một URL từ xa do người dùng cung cấp mà không kiểm tra, xác thực đúng cách. Kẻ tấn công có thể lợi dụng điều này để buộc máy chủ gửi các yêu cầu tùy ý đến các hệ thống nội bộ (mạng nội bộ, dịch vụ metadata trên cloud) hoặc bên thứ ba, qua mặt các tường lửa. Điều này có thể dẫn đến việc đánh cắp dữ liệu nhạy cảm, quét và tấn công các hệ thống nội bộ[8].

### ***1.2.3. Ứng dụng của OWASP Top 10 trong thực tiễn và Secure SDLC***

Việc nắm vững OWASP Top 10 không chỉ có giá trị lý thuyết mà còn có ứng dụng thực tiễn sâu rộng trong quy trình phát triển phần mềm an toàn (Secure SDLC). cụ thể là:

Trong giai đoạn Yêu cầu: Danh sách OWASP Top 10 cung cấp một bộ kiểm tra (checklist) các yêu cầu phi chức năng về bảo mật. Các yêu cầu như "Ứng dụng phải chống được các lỗ hổng Injection" hay "Phải mã hóa dữ liệu nhạy cảm để ngăn ngừa Cryptographic Failures" có thể được đưa vào tài liệu dự án ngay từ đầu.

Trong giai đoạn Thiết kế: "Insecure Design" - một hạng mục mới trong Top 10 2021 - nhấn mạnh trực tiếp tầm quan trọng của giai đoạn này. Các hoạt động như mô hình hóa mối đe dọa (threat modeling) có thể được hướng dẫn bởi các rủi ro trong OWASP Top 10 để tìm ra các điểm yếu tiềm tàng trong kiến trúc.

Trong giai đoạn Phát triển: OWASP cung cấp các cheat sheet và hướng dẫn lập trình an toàn cụ thể để tránh các lỗ hổng như Injection, Broken Access Control hay Cryptographic Failures. Các công cụ SAST và SCA có thể được cấu hình để phát hiện các mẫu mã tương ứng với 10 rủi ro này.

Trong giai đoạn Kiểm thử: Đây là ứng dụng phổ biến nhất. Các đội kiểm thử xâm nhập (penetration testing) luôn sử dụng OWASP Top 10 như một khung kiểm thử chính, đảm bảo ứng dụng không mắc phải 10 lỗ hổng nguy hiểm nhất trước khi lên môi trường production.

Như vậy, OWASP Top 10 không chỉ là một danh sách các lỗ hổng. Nó là một công cụ chiến lược, một ngôn ngữ chung cho toàn bộ các bên liên quan (từ lãnh đạo, quản lý dự án, kiến trúc sư, nhà phát triển đến kiểm thử viên) để nói về và quản lý rủi ro bảo mật, qua đó đóng vai trò then chốt trong việc xây dựng một quy trình Secure SDLC vững chắc.

## 1.3. Các phương pháp kiểm thử bảo mật ứng dụng

### 1.3.1. Khái niệm và mục tiêu của kiểm thử bảo mật ứng dụng

Kiểm thử bảo mật ứng dụng (Application Security Testing – AST) là quá trình chủ động xác định, phân tích và đánh giá các lỗ hổng, điểm yếu và rủi ro bảo mật trong một ứng dụng phần mềm[9], [14]. Khác với kiểm thử chức năng truyền thống (vốn chỉ tập trung vào việc xác minh các yêu cầu nghiệp vụ), kiểm thử bảo mật hướng đến việc trả lời câu hỏi: “Liệu kẻ tấn công có thể khai thác ứng dụng này để gây hại hay không?”.

Mục tiêu cốt lõi của kiểm thử bảo mật ứng dụng bao gồm:

- Phát hiện lỗ hổng trước khi kẻ tấn công phát hiện: Đây là mục tiêu phòng ngừa quan trọng nhất, giúp giảm thiểu rủi ro và chi phí khắc phục.
- Đánh giá mức độ nghiêm trọng và khả năng khai thác: Không phải lỗ hổng nào cũng nguy hiểm như nhau. Kiểm thử bảo mật giúp phân loại và ưu tiên xử lý dựa trên bối cảnh thực tế.
- Tuân thủ các tiêu chuẩn và quy định: Nhiều ngành (tài chính, y tế) yêu cầu bắt buộc phải có báo cáo kiểm thử bảo mật định kỳ.
- Xây dựng văn hóa bảo mật: Kết quả kiểm thử cung cấp phản hồi cụ thể cho đội ngũ phát triển, giúp họ học hỏi và tránh lặp lại các lỗi tương tự.

Trong thực tế, không có một phương pháp kiểm thử duy nhất nào có thể bao quát toàn bộ các mối đe dọa. Do đó, các tổ chức thường áp dụng chiến lược kết hợp nhiều phương pháp khác nhau, tạo thành một hệ thống phòng thủ theo lớp (defense in depth) cho ứng dụng.

- Phân loại các phương pháp kiểm thử bảo mật ứng dụng



Dựa trên thời điểm thực hiện, cách thức tiếp cận và phạm vi tác động, các phương pháp AST có thể được phân thành các nhóm chính sau đây.

– Kiểm thử tĩnh (Static Testing)

- Kiểm thử tĩnh được thực hiện trên mã nguồn, bytecode hoặc tệp nhị phân của ứng dụng mà không cần chạy chương trình. Nó phân tích cấu trúc, luồng dữ liệu và các mẫu mã để tìm ra các lỗi bảo mật điển hình.
- SAST (Static Application Security Testing): Đây là công cụ phân tích tĩnh chính thống. SAST quét toàn bộ cây mã nguồn, phát hiện các lỗ hổng như SQL Injection, XSS, lỗi buffer overflow, v.v. Ưu điểm lớn nhất của SAST là có thể áp dụng từ rất sớm (ngay khi viết xong một module code), tích hợp vào IDE hoặc pipeline CI/CD. Tuy nhiên, SAST có thể sinh ra nhiều cảnh báo sai (false positives) và không phát hiện được các lỗ hổng liên quan đến cấu hình môi trường khi runtime.
- SCA (Software Composition Analysis): Là một dạng kiểm thử tĩnh chuyên biệt, tập trung vào việc phân tích các thư viện mã nguồn mở (third-party libraries) mà dự án sử dụng. SCA so sánh danh sách các thành phần này với các cơ sở dữ liệu lỗ hổng công khai (như CVE, NVD) để phát hiện các phiên bản cũ, có lỗ hổng đã biết. Trong bối cảnh hiện đại (ứng dụng có tới 80% mã nguồn từ thư viện bên ngoài), SCA trở nên cực kỳ quan trọng để quản lý rủi ro chuỗi cung ứng phần mềm (software supply chain), liên quan mật thiết đến rủi ro A06:2021 (Vulnerable and Outdated Components).

– Kiểm thử động (Dynamic Testing)

- Kiểm thử động được thực hiện trên ứng dụng đang chạy (runtime), thường trong môi trường staging hoặc production. Phương pháp này mô phỏng hành vi của kẻ tấn công từ bên ngoài, gửi các yêu cầu (requests) độc hại và phân tích phản hồi (responses).
- DAST (Dynamic Application Security Testing): Công cụ DAST hoạt động như một “tin tặc tự động”, quét ứng dụng web từ ngoài vào, không cần biết mã nguồn bên trong. Nó thử các tấn công như injection, XSS, CSRF, cấu hình sai, v.v. DAST có ưu điểm là ít cảnh báo sai hơn SAST, phát hiện được các vấn đề cấu hình runtime, và có thể sử dụng ở giai đoạn kiểm thử chấp nhận (UAT). Nhược điểm là chỉ phát hiện được những lỗ hổng mà nó đã có sẵn kịch bản tấn công, quá trình quét có thể chậm và đôi khi làm thay đổi dữ liệu trên hệ thống (cần cẩn thận khi quét production).
- IAST (Interactive Application Security Testing): Là sự kết hợp giữa SAST và DAST. IAST sử dụng các “agent” cài đặt bên trong ứng dụng (ví dụ: trong máy chủ Java/.NET) để giám sát và phân tích lưu lượng truy cập trong thời gian thực. Khi DAST hoặc một người dùng thực tế gửi yêu cầu, IAST sẽ nhìn vào luồng dữ liệu, truy vấn cơ sở dữ liệu, lệnh gọi API để xác định chính xác vị trí lỗ hổng trong mã nguồn. IAST cho độ chính xác rất cao (ít false positives), cung cấp bằng chứng rõ ràng cả về yêu cầu và dòng code, nhưng cần có khả năng cài đặt agent và tương thích với ứng dụng.

– Kiểm thử xâm nhập (Penetration Testing)

- Khác với các công cụ tự động, kiểm thử xâm nhập (Penetration Testing hay Pentest) là quá trình thủ công, có chủ đích, do các chuyên gia bảo mật (pentester) thực hiện. Họ sử dụng tổng hợp các công cụ (bao gồm cả SAST, DAST, và các kỹ thuật khai thác thủ công) để tìm cách xâm nhập vào ứng dụng và hệ thống.
- Điểm mạnh: Pentest phát hiện được các lỗ hổng phức tạp, logic nghiệp vụ (business logic flaws), hoặc các lỗ hổng kết hợp nhiều bước mà công cụ tự động khó nhận ra. Nó cũng đánh giá được mức độ ảnh hưởng thực tế của một lỗ hổng (ví dụ: liệu có thể leo thang đặc quyền để lấy toàn bộ cơ sở dữ liệu hay không).
- Điểm yếu: Chi phí cao, phụ thuộc nhiều vào kỹ năng của pentester, và chỉ thực hiện được tại một thời điểm nhất định (không liên tục). Do đó, pentest thường được thực hiện định kỳ (hàng quý, hàng năm) hoặc trước các sự kiện quan trọng (ra mắt sản phẩm lớn, kiểm thử tuân thủ).

– Bảo vệ thời gian thực (RASP – Runtime Application Self-Protection)

- RASP là một công nghệ bảo mật hiện đại, không chỉ kiểm thử mà còn bảo vệ ứng dụng ngay khi đang chạy. RASP được cài đặt bên trong ứng dụng hoặc máy chủ ứng dụng. Nó phân tích liên tục các hành vi, luồng dữ liệu, tham số đầu vào, và có khả năng tự động chặn các yêu cầu độc hại trong thời gian thực mà không cần sự can thiệp của con người. RASP được coi là lớp phòng thủ cuối cùng, bổ sung cho các phương pháp kiểm thử truyền thống.

### ***1.3.2. Chiến lược kết hợp và vị trí trong Secure SDLC***

Trong một Secure SDLC hoàn chỉnh, các phương pháp kiểm thử bảo mật không được sử dụng riêng rẽ mà được bố trí một cách có chiến lược ở các giai đoạn khác nhau:

Giai đoạn Phát triển (Development): Tích hợp SAST và SCA vào pipeline CI/CD, thực hiện sau mỗi lần commit code. Kết quả được trả về ngay lập tức để nhà phát triển sửa lỗi. Đồng thời, có thể áp dụng IAST trên môi trường dev để kiểm tra các tính năng mới.

Giai đoạn Kiểm thử (Testing): Triển khai DAST quét toàn diện trên môi trường staging, kết hợp với các phiên kiểm thử xâm nhập thủ công định kỳ hoặc trước khi phát hành major release.

Giai đoạn Vận hành (Operations): RASP được kích hoạt trong môi trường production để bảo vệ ứng dụng trước các tấn công chưa được biết đến (zero-day). Đồng thời, các ghi nhật ký (logs) từ kiểm thử động và RASP cũng được đưa vào hệ thống giám sát bảo mật (gắn với rủi ro A09:2021 về giám sát).

Sự kết hợp “SAST + DAST + Pentest + RASP” theo một lộ trình phù hợp được xem là thực hành tiên tiến nhất, cho phép phát hiện lỗ hổng ở cả ba tầng: mã nguồn (tĩnh), hành vi (động) và ngữ cảnh nghiệp vụ (thủ công), đồng thời có khả năng phản vệ khi đã triển khai. Việc lựa chọn cụ thể các phương pháp nào phụ thuộc vào nguồn lực, mức độ rủi ro chấp nhận được và loại ứng dụng của từng dự án.

## 1.4. Bảo mật hạ tầng và môi trường hiện đại

### 1.4.1. Khái niệm và bối cảnh

Trong các mô hình phát triển phần mềm truyền thống, hạ tầng thường được hiểu là các máy chủ vật lý hoặc máy ảo đặt trong trung tâm dữ liệu, với ranh giới mạng rõ ràng và ít thay đổi. Tuy nhiên, với sự bùng nổ của điện toán đám mây (cloud computing), container, hệ thống orchestration (như Kubernetes) và các pipeline triển khai liên tục (CI/CD), khái niệm “hạ tầng” đã thay đổi hoàn toàn.

Bảo mật hạ tầng và môi trường hiện đại là tập hợp các nguyên tắc, công cụ và quy trình nhằm bảo vệ toàn bộ các thành phần của hệ thống – từ tầng vật lý (hoặc ảo hóa) lên đến tầng ứng dụng – trong bối cảnh hạ tầng được định nghĩa bằng mã (Infrastructure as Code – IaC), có tính đàn hồi (elastic), thay đổi liên tục và phân tán về mặt địa lý. Điểm đặc biệt của bảo mật hạ tầng hiện đại là nó không thể tách rời khỏi quy trình phát triển và vận hành (DevOps), đòi hỏi mô hình trách nhiệm chung (shared responsibility) giữa nhà cung cấp dịch vụ đám mây và khách hàng, cũng như áp dụng các nguyên lý của “DevSecOps” – tích hợp bảo mật vào mọi khía cạnh của pipeline và hạ tầng.

### 1.4.2. Các thành phần chính và rủi ro bảo mật đặc thù

Môi trường hiện đại thường bao gồm bốn lớp hạ tầng chính, mỗi lớp đều có bề mặt tấn công riêng [10].

- Container (Docker, containerd, ...)
  - Container là công nghệ ảo hóa cấp hệ điều hành, cho phép đóng gói ứng dụng và các thư viện phụ thuộc vào một đơn vị nhẹ, di động. Các rủi ro bảo mật đặc thù của container bao gồm:
    - Hình ảnh (image) không an toàn: Hình ảnh container có thể chứa các thành phần lỗi thời, có lỗ hổng (liên quan đến SCA), hoặc được tạo từ các nguồn không đáng tin cậy.
    - Leak thông tin nhạy cảm trong image: Mật khẩu, khóa API hoặc token thường bị lưu cứng (hardcoded) vào các layer của image.
    - Escalation từ container lên host: Nếu container được chạy với quyền root hoặc các capability không cần thiết (ví dụ: CAP\_SYS\_ADMIN), kẻ tấn công có thể “thoát container” và kiểm soát toàn bộ node host.
    - Cấu hình runtime sai: Cho phép container chạy ở chế độ privileged, mount các thư mục nhạy cảm của host (như /docker.sock) hoặc chia sẻ namespace kernel không an toàn.
- Hệ thống orchestration (Kubernetes – K8s) [11]
  - Kubernetes là hệ thống quản lý container phổ biến nhất, tự động hóa việc triển khai, mở rộng và vận hành các container. Các rủi ro điển hình trong K8s:

- API server không được bảo vệ: API server là “bộ não” của K8s. Nếu để lộ ra ngoài hoặc không có xác thực/ủy quyền mạnh, kẻ tấn công có thể gửi các lệnh tùy ý (tạo pod độc hại, đánh cắp secret).
  - Cấu hình RBAC (Role-Based Access Control) quá rộng: Gán quyền quá lớn cho một service account hoặc người dùng, dẫn đến leo thang đặc quyền.
  - Secret không được mã hóa hoặc quản lý kém: K8s có đối tượng Secret để lưu thông tin nhạy cảm, nhưng nếu không bật mã hóa etcd, secret sẽ tồn tại dưới dạng plaintext.
  - Các lỗ hổng trong thành phần K8s: Chính các thành phần như kubelet, kube-proxy, hoặc container runtime có thể có CVE riêng, cần được cập nhật thường xuyên.
- Điện toán đám mây (AWS, Azure, GCP)
- Môi trường cloud cung cấp cơ sở hạ tầng dưới dạng dịch vụ (IaaS), nền tảng (PaaS) và phần mềm (SaaS). Các rủi ro đặc thù của cloud bao gồm:
  - Cấu hình sai tài khoản và quyền IAM (Identity and Access Management): Đây là rủi ro phổ biến nhất, ví dụ như bucket S3 để công khai (public), cấp quyền “administrator” cho một người dùng chỉ cần đọc log, hoặc không xoay vòng (rotate) key truy cập.
  - Phơi lộ các endpoint quản lý: Các dịch vụ như RDS, Redis, Elasticsearch được cấu hình mở cổng ra internet mà không có giới hạn IP.
  - Tài nguyên bị chiếm dụng cho tấn công (cryptojacking, DDoS): Kẻ tấn công khi khai thác được lỗ hổng có thể dựng các container hoặc VM để đào tiền ảo hoặc tấn công mạng, gây tổn kém chi phí.
  - Thiếu giám sát và cảnh báo cho các hoạt động bất thường trên cloud (ví dụ: một instance bất ngờ thay đổi security group).
- Infrastructure as Code (IaC) và Pipeline CI/CD[12]
- Trong môi trường hiện đại, hạ tầng được mô tả bằng mã (Terraform, CloudFormation, Ansible) và được triển khai qua pipeline CI/CD (Jenkins, GitLab CI, GitHub Actions). Các rủi ro liên quan đến IaC và pipeline:
  - Mã IaC chứa secret hoặc cấu hình không an toàn: Ví dụ, một file Terraform có thể khai báo một bucket S3 public, hoặc chứa mật khẩu cơ sở dữ liệu dưới dạng plaintext.
  - Pipeline bị tấn công (supply chain attack): Kẻ tấn công có thể chiếm quyền kiểm soát server CI/CD, tiêm mã độc vào quá trình build, hoặc thay thế artifact bằng phiên bản độc hại.
  - Thiếu kiểm tra bảo mật trên các template IaC: Trước khi triển khai, template cần được phân tích tĩnh (ví dụ bằng Checkov, tfsec) để phát hiện các cấu hình sai từ sớm (shift-left).

### 1.4.3. Các nguyên tắc và biện pháp bảo mật cho hạ tầng hiện đại

Để đối phó với các rủi ro trên, các tổ chức áp dụng một bộ nguyên tắc và công cụ thường được gọi là "bảo mật cho cloud-native".

- Nguyên tắc "Zero Trust" và "Least Privilege" mở rộng
  - Zero Trust (Không tin tưởng tuyệt đối): Không có thành phần nào (dù nằm trong mạng nội bộ) được mặc định tin cậy. Mọi yêu cầu truy cập giữa các container, service, hoặc từ người dùng đến API đều phải được xác thực và ủy quyền. Điều này dẫn đến việc sử dụng mạng mesh (service mesh như Istio, Linkerd) với mTLS (mutual TLS) để mã hóa và xác thực lẫn nhau.
  - Least Privilege (Tối thiểu đặc quyền): Áp dụng từ cấp độ IAM cloud đến RBAC trong K8s và quyền của container (không chạy container với user root, chỉ gán các Linux capability thực sự cần thiết).
- Quét và kiểm soát hình ảnh container (Container Image Scanning)
  - Đây là hoạt động bắt buộc trong pipeline CI/CD: sử dụng các công cụ như Trivy, Clair, Snyk, Docker Scout để quét các layer của image, phát hiện lỗ hổng trong các gói hệ điều hành và thư viện ứng dụng. Quy trình lý tưởng bao gồm:
    - Quét tĩnh trên image ngay sau khi build (trước khi push lên registry).
    - Chỉ cho phép các image có mức độ nghiêm trọng dưới ngưỡng (ví dụ: không có lỗ hổng Critical/High) được triển khai.
    - Quét lại định kỳ các image đang chạy trong production vì các CVE mới có thể được công bố sau đó.
- Cấu hình bảo mật cho Kubernetes (K8s Security Best Practices)
  - Sử dụng Pod Security Standards (PSS) hoặc Pod Security Admission (PSA): Áp dụng các chế độ bảo mật được đề xuất (baseline, restricted) để ngăn các pod chạy với quyền nguy hiểm.
  - Network Policies: Định nghĩa các chính sách mạng mặc định deny-all, chỉ cho phép lưu lượng cần thiết giữa các pod và namespace.
  - Quản lý secret bằng các công cụ ngoài Kubernetes: Sử dụng các giải pháp như HashiCorp Vault, AWS Secrets Manager, hoặc Azure Key Vault, kết hợp với CSI driver để mount secret vào pod mà không cần lưu trong etcd.
  - Cập nhật và vá lỗi thường xuyên cho cluster K8s (control plane và worker nodes).
- Bảo mật pipeline CI/CD (Pipeline Security)
  - Không lưu secret trong mã nguồn: Sử dụng secret manager của CI/CD (GitLab CI variables, GitHub Secrets, Jenkins Credentials).
  - Ký và xác minh artifact: Dùng các cơ chế như sigstore, cosign để ký các image container, đảm bảo tính toàn vẹn và nguồn gốc.

- Kiểm tra IaC trước khi deploy: Tích hợp các công cụ như Checkov, tfsec, Terrascan vào pipeline để phát hiện cấu hình sai.
- Phân quyền truy cập pipeline tối thiểu: Chỉ những người dùng hoặc service account thực sự cần mới có quyền trigger hoặc sửa pipeline.

#### ***1.4.4. Mối quan hệ với Secure SDLC và kiểm thử bảo mật***

Bảo mật hạ tầng hiện đại không thể tách rời khỏi các nội dung đã trình bày trước đó:

- Với Secure SDLC (mục 1.1): Các hoạt động bảo mật hạ tầng (quét image, kiểm tra IaC, cấu hình K8s) cần được tích hợp như những "gate" (cửa kiểm soát) trong pipeline CI/CD, thuộc giai đoạn Triển khai và Bảo trì. Đồng thời, các yêu cầu về hạ tầng (ví dụ: cần hỗ trợ mTLS, phải chạy trong môi trường không root) phải được đưa vào giai đoạn Thiết kế.
- Với OWASP Top 10 (mục 1.2): Nhiều rủi ro trong OWASP có nguyên nhân sâu xa từ hạ tầng. Ví dụ: A05:2021 Security Misconfiguration thường đến từ việc cấu hình sai cloud hoặc container; A08:2021 Software and Data Integrity Failures liên quan đến việc không ký và kiểm tra artifact trong pipeline; A10:2021 SSRF có thể được giảm nhẹ bằng cách kiểm soát egress network policy trong K8s.
- Với kiểm thử bảo mật (mục 1.3): Các công cụ DAST truyền thống cần được điều chỉnh để quét các ứng dụng chạy trong container hoặc trên cloud, đồng thời xuất hiện các loại kiểm thử mới như kiểm thử cấu hình hạ tầng (infrastructure testing) với các công cụ như kube-hunter, kube-bench, và kiểm thử IaC.

## CHƯƠNG 2. NỘI DUNG NGHIÊN CỨU

### 2.1. OWASP ZAP (Zed Attack Proxy)

#### 2.1.1. Khái niệm và mục đích

OWASP ZAP (Zed Attack Proxy) là một công cụ kiểm thử bảo mật ứng dụng web mã nguồn mở, được phát triển và duy trì bởi tổ chức OWASP (Open Worldwide Application Security Project). Được biết đến như một trong những công cụ DAST (Dynamic Application Security Testing) phổ biến và mạnh mẽ nhất hiện nay, ZAP được thiết kế để phát hiện các lỗ hổng bảo mật trong ứng dụng web khi ứng dụng đang chạy.

#### 2.1.2. Kiến trúc và nguyên lý hoạt động

ZAP hoạt động dựa trên cơ chế "man-in-the-middle proxy" (proxy trung gian). Cụ thể, nó được đặt giữa trình duyệt của người kiểm thử và ứng dụng web mục tiêu, cho phép chặn, kiểm tra, sửa đổi tất cả các yêu cầu (request) và phản hồi (response) HTTP/HTTPS được trao đổi. Kiến trúc này cho phép ZAP can thiệp sâu vào luồng dữ liệu, đóng vai trò như một "người quan sát toàn năng" để phân tích hành vi và tìm kiếm điểm yếu.

Nguyên lý hoạt động cốt lõi của ZAP bao gồm hai chế độ quét chính:

- Quét thụ động (Passive Scanning): Ở chế độ này, ZAP chỉ lắng nghe và phân tích tất cả các request/response đi qua proxy mà không thay đổi hoặc gửi thêm bất kỳ yêu cầu nào. Nó an toàn và không gây hại cho ứng dụng, chủ yếu được sử dụng để rà soát các vấn đề về cấu hình bảo mật, header HTTP thiếu an toàn hoặc phát hiện thông tin nhạy cảm bị rò rỉ trên giao diện người dùng.
- Quét chủ động (Active Scanning): Đây là chế độ tấn công mô phỏng. Dựa trên danh sách các điểm cuối (endpoints) đã được thu thập (thông qua quá trình thu thập dữ liệu - spidering), ZAP sẽ tự động gửi một loạt các request được thiết kế đặc biệt (các payload tấn công) đến ứng dụng để kiểm tra các lỗ hổng như SQL Injection (SQLi), Cross-Site Scripting (XSS), Path Traversal, v.v..

#### 2.1.3. Các tính năng chính

Hỗ trợ đa nền tảng và dễ sử dụng: ZAP có sẵn cho Windows, Linux và macOS, với giao diện đồ họa (GUI) thân thiện, phù hợp cho cả người mới bắt đầu lẫn chuyên gia.

Tích hợp CI/CD mạnh mẽ: ZAP cung cấp API REST và các script tự động hóa, cho phép tích hợp sâu vào các pipeline CI/CD (như Jenkins, GitLab CI, GitHub Actions) để thực hiện kiểm thử bảo mật tự động trong quy trình phát triển, minh họa rõ nét cho mô hình DevSecOps đã được đề cập ở Chương 1.

Khả năng mở rộng cao: Là một dự án mã nguồn mở với cộng đồng mạnh mẽ, ZAP cho phép người dùng cài đặt thêm các add-on từ "ZAP Marketplace" hoặc tự viết

script tùy chỉnh bằng nhiều ngôn ngữ (Python, JavaScript) để đáp ứng các nhu cầu kiểm thử đặc biệt.

Mô phỏng tấn công đa dạng: Công cụ tích hợp sẵn các kỹ thuật fuzzing (gửi dữ liệu đầu vào bất thường) và forced browsing (duyệt cưỡng bức) để phát hiện các thư mục, file ẩn hoặc các điểm yếu logic nghiệp vụ.

#### ***2.1.4. Vai trò trong bối cảnh nghiên cứu***

Trong nghiên cứu này, OWASP ZAP được lựa chọn như một công cụ DAST đại diện cho lớp kiểm thử tự động, hiệu quả và miễn phí. Nó là công cụ chính để thực hiện các thử nghiệm quét tự động nhằm phát hiện các lỗ hổng phổ biến theo danh mục OWASP Top 10, đồng thời là công cụ để so sánh đối chiếu kết quả với các phương pháp thủ công (sử dụng Burp Suite).

## **2.2. Burp Suite (Community Edition)**

### ***2.2.1. Khái niệm và phân khúc***

Burp Suite, phát triển bởi PortSwigger, là một nền tảng kiểm thử bảo mật ứng dụng web được công nhận rộng rãi như một tiêu chuẩn trong ngành, đặc biệt trong lĩnh vực kiểm thử xâm nhập (penetration testing) chuyên nghiệp[13]. Burp Suite có ba phiên bản: Community (miễn phí), Professional (trả phí, \$449/năm/người dùng) và Enterprise (dành cho doanh nghiệp). Trong phạm vi nghiên cứu này, nhóm sử dụng Burp Suite Community Edition, một lựa chọn lý tưởng cho mục đích học tập và thực hành kiểm thử thủ công.

### ***2.2.2. Cơ chế hoạt động và các thành phần chính***

- Tương tự như ZAP, Burp Suite cũng hoạt động dựa trên cơ chế proxy trung gian. Tuy nhiên, điểm mạnh vượt trội của Burp Suite (kể cả bản Community) nằm ở bộ công cụ (toolset) hỗ trợ kiểm thử thủ công cực kỳ mạnh mẽ và tinh tế, bao gồm:
- Proxy (Bộ chặn): Cho phép người dùng chặn, xem xét và chỉnh sửa từng request/response theo thời gian thực (real-time), giúp hiểu rõ chi tiết cách ứng dụng xử lý dữ liệu.
- Repeater (Bộ lặp): Công cụ này cho phép người dùng lấy một request bất kỳ từ proxy, sửa đổi tham số, và gửi đi lặp lại nhiều lần để quan sát phản hồi của máy chủ. Đây là công cụ không thể thiếu để khai thác thủ công các lỗ hổng như SQLi hoặc XSS.
- Intruder (Bộ xâm nhập): Đây là công cụ tự động hóa bán phần. Người dùng định nghĩa một request mẫu, đánh dấu các vị trí muốn tấn công (ví dụ: trường username và password), cung cấp một danh sách các payload (từ điển tấn công), và Intruder sẽ tự động gửi hàng loạt request và so sánh kết quả trả về. Trong bản Community Edition, chức năng này có giới hạn về tốc độ (throttled) nhưng vẫn đủ để thực hiện các bài kiểm thử quy mô nhỏ.



- Decoder (Bộ giải mã): Hỗ trợ mã hóa/giải mã dữ liệu ở nhiều định dạng khác nhau (Base64, URL, HTML...), rất hữu ích khi phân tích token hoặc các tham số phức tạp.

### **2.2.3. Thế mạnh so với OWASP ZAP (trong bản miễn phí)**

Burp Suite Community Edition có một số khác biệt chiến lược so với ZAP:

- Tập trung vào kiểm thử thủ công: Trong khi ZAP mạnh về quét tự động, Burp Community lại xuất sắc trong việc can thiệp và phân tích luồng dữ liệu ở cấp độ chi tiết nhất. Nó giúp người kiểm thử hiểu rõ bản chất của lỗ hổng, thay vì chỉ trông chờ vào báo cáo từ máy móc.
- Độ ổn định và phổ biến: Là công cụ tiêu chuẩn, cộng đồng sử dụng Burp Suite rất lớn, tài liệu học tập và các bài hướng dẫn khai thác (walkthrough) rất phong phú.
- Hiệu năng ưu việt cho các tác vụ thủ công: Giao diện và luồng công việc (workflow) được tối ưu cho người dùng là chuyên gia bảo mật, thao tác nhanh và trực quan.

### **2.2.4. Vai trò trong bối cảnh nghiên cứu**

Trong nghiên cứu này, Burp Suite Community Edition được sử dụng như một công cụ đối chiếu và xác thực. Nó sẽ được dùng để kiểm tra lại bằng tay các lỗ hổng mà OWASP ZAP báo cáo, đồng thời khai thác các lỗ hổng phức tạp về logic nghiệp vụ mà công cụ tự động có thể bỏ sót. Việc kết hợp cả hai công cụ thể hiện một chiến lược kiểm thử bảo mật hiệu quả và toàn diện như đã trình bày trong mục 1.3.

## **2.3. Môi trường mục tiêu: DVWA (Damn Vulnerable Web App)**

### **2.3.1. Khái niệm và mục đích**

DVWA (Damn Vulnerable Web Application) là một ứng dụng web được cố tình xây dựng với rất nhiều lỗ hổng bảo mật, viết bằng ngôn ngữ PHP và sử dụng cơ sở dữ liệu MySQL/MariaDB. Mục đích chính của DVWA là cung cấp một môi trường hợp pháp, an toàn và có kiểm soát để các chuyên gia bảo mật, sinh viên và lập trình viên thực hành các kỹ năng kiểm thử xâm nhập, khai thác lỗ hổng và học cách bảo vệ ứng dụng web.[6]

### **2.3.2. Các tính năng chính phục vụ học tập và nghiên cứu**

Kho lưu trữ các lỗ hổng phổ biến: DVWA tích hợp sẵn hầu hết các lỗ hổng nguy hiểm nhất theo OWASP Top 10, bao gồm: SQL Injection, Command Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), File Inclusion, File Upload, Brute Force....

Các mức độ khó khác nhau (Low, Medium, High, Impossible): Đây là tính năng đặc sắc nhất của DVWA. Mỗi lỗ hổng được cung cấp ở bốn cấp độ bảo mật khác nhau:

- Low: Không có cơ chế phòng vệ nào, dễ khai thác nhất dành cho người mới bắt đầu.

- Medium/Hight: Các biện pháp bảo vệ như mã hóa, lọc đầu vào (input validation) được thêm vào, đòi hỏi kỹ thuật khai thác phức tạp hơn.
- Impossible: Mã nguồn được viết một cách an toàn tuyệt đối, không còn lỗ hổng. Cấp độ này đóng vai trò như một bài mẫu (reference) về lập trình an toàn.
- Môi trường an toàn và biệt lập: DVWA tuyệt đối không được triển khai trên các máy chủ công cộng hay có kết nối Internet. Nó được thiết kế để chạy trên localhost hoặc trong các máy ảo (VirtualBox, VMware) với chế độ mạng NAT. Điều này đảm bảo việc thực hành khai thác lỗ hổng không gây ảnh hưởng ra bên ngoài.

### ***2.3.3. Vai trò trong bối cảnh nghiên cứu***

DVWA đóng vai trò là "vật thử nghiệm" (testbed) lý tưởng. Nó cho phép nhóm nghiên cứu kiểm tra toàn diện các tính năng của OWASP ZAP và Burp Suite trong một môi trường được kiểm soát hoàn toàn. Bằng cách thay đổi các mức độ khó, nhóm có thể đánh giá chính xác khả năng của từng công cụ trong việc phát hiện cùng một lỗ hổng nhưng ở các cấp độ phòng vệ khác nhau, từ đó đưa ra những so sánh thực nghiệm có giá trị.

## **2.4. Công cụ kiểm tra Container và CI/CD**

### ***2.4.1. Bối cảnh: Bảo mật trong môi trường hiện đại***

Như đã phân tích trong phần bảo mật hạ tầng và môi trường hiện đại, việc sử dụng container (Docker) và các pipeline CI/CD đã trở thành tiêu chuẩn trong phát triển phần mềm[12]. Điều này kéo theo những rủi ro mới, đặc biệt là từ các thành phần bên thứ ba (third-party libraries) và các cấu hình sai trong mã nguồn của hạ tầng.

### ***2.4.2. Trivy: Công cụ quét bảo mật đa năng cho Container và IaC***

Để giải quyết các thách thức trên, nhóm nghiên cứu lựa chọn Trivy - một công cụ quét bảo mật mã nguồn mở phổ biến nhất hiện nay, do công ty Aqua Security phát triển. Trivy được thiết kế để "tìm lỗ hổng (CVE) và cấu hình sai (misconfigurations) trên các kho lưu trữ mã nguồn, artifacts nhị phân, hình ảnh container và các cluster Kubernetes".

### ***2.4.3. Các tính năng chính và nguyên lý***

Quét lỗ hổng trên Container Image (Container Security): Trivy so sánh danh sách các gói phần mềm (packages) được cài đặt bên trong một Docker image với các cơ sở dữ liệu lỗ hổng (CVE databases) khổng lồ, từ đó phát hiện và báo cáo các thành phần đã cũ, có lỗ hổng bảo mật đã biết. Đây là ứng dụng trực tiếp của khái niệm SCA (Software Composition Analysis) trong môi trường container[7].

Quét cấu hình Infrastructure as Code (IaC Security): Trivy có khả năng phân tích các file mã nguồn định nghĩa hạ tầng như Terraform, Kubernetes YAML, Dockerfile, CloudFormation để phát hiện các cấu hình sai. Ví dụ: một container được cấp quyền privileged: true, một cổng (port) nhạy cảm bị mở ra công chúng, hoặc một bucket S3 được cấu hình public.

Tích hợp CI/CD: Trivy được sinh ra để phục vụ cho tự động hóa. Nó có thể được nhúng dễ dàng như một bước (step) trong pipeline CI/CD (GitLab CI, GitHub Actions, Jenkins...). Thông thường, nó sẽ được cấu hình để "làm gãy" (fail) pipeline nếu phát hiện lỗ hổng ở mức Nghiêm trọng (Critical) hoặc Cao (High), ngăn chặn các image mất an toàn được đưa lên production.

Tốc độ và tính đa năng: Trivy được đánh giá cao nhờ tốc độ quét nhanh (chỉ mất vài giây) và hỗ trợ nhiều loại đối tượng quét khác nhau, bao gồm cả hệ thống tập tin (filesystem) và kho lưu trữ Git.

## **2.5. SQL Injection – Cơ sở lý thuyết và phương pháp phòng chống**

### **2.5.1. Khái niệm**

SQL Injection (SQLi) là một kỹ thuật tấn công khai thác lỗ hổng bảo mật cho phép kẻ tấn công can thiệp vào các truy vấn mà ứng dụng gửi tới cơ sở dữ liệu. Lỗ hổng xảy ra khi dữ liệu đầu vào từ người dùng không được kiểm tra, lọc hoặc xử lý đúng cách trước khi được nhúng trực tiếp vào câu lệnh SQL. Hậu quả có thể bao gồm: đọc, sửa, xóa dữ liệu trái phép, thậm chí thực thi lệnh điều khiển hệ điều hành trên máy chủ cơ sở dữ liệu. Đây là một trong những lỗ hổng nguy hiểm nhất và đã được xếp ở vị trí A03:2021 – Injection trong OWASP Top 10 (phiên bản 2021).

### **2.5.2. Các dạng tấn công SQL Injection phổ biến**

- In-band SQLi (Classic): Kẻ tấn công nhận kết quả trả về qua cùng kênh giao tiếp. Gồm hai dạng phụ:
- Union-based: Sử dụng toán tử UNION để kết hợp kết quả từ nhiều truy vấn, lấy dữ liệu từ các bảng khác.
- Error-based: Dựa vào thông báo lỗi chi tiết từ cơ sở dữ liệu để suy ra cấu trúc và nội dung của bảng.
- Blind SQLi: Không thấy trực tiếp dữ liệu hay thông báo lỗi. Kẻ tấn công phải hỏi máy chủ những câu hỏi đúng/sai (Boolean-based) hoặc dựa vào độ trễ phản hồi (Time-based).
- Out-of-band SQLi: Dữ liệu được gửi qua một kênh khác (ví dụ DNS hoặc HTTP request tới máy chủ do kẻ tấn công kiểm soát). Thường dùng khi kết quả không thể hiển thị trực tiếp.

### **2.5.3. Nguyên tắc phòng chống SQL Injection**

- Sử dụng truy vấn có tham số (Prepared Statements / Parameterized Queries): Đây là biện pháp hiệu quả nhất. Câu lệnh SQL được tạo sẵn với các placeholder, sau đó dữ liệu đầu vào được truyền vào qua tham số, đảm bảo không bị thực thi như mã lệnh.
- Xác thực đầu vào (Input Validation): Kiểm tra dữ liệu nhập về kiểu, độ dài, định dạng. Áp dụng danh sách trắng (whitelist) thay vì danh sách đen (blacklist) khi có thể.

- Thoát ký tự đặc biệt (Escaping): Sử dụng hàm thoát dành riêng cho cơ sở dữ liệu đang dùng (ví dụ `mysqli_real_escape_string()` trong PHP). Tuy nhiên, không an toàn tuyệt đối bằng Prepared Statements.
- Nguyên tắc đặc quyền tối thiểu (Least Privilege): Tài khoản kết nối cơ sở dữ liệu của ứng dụng chỉ nên có quyền cần thiết nhất (SELECT, INSERT, UPDATE, DELETE trên đúng bảng), không cấp quyền DROP, ALTER hay FILE.
- Ẩn thông tin lỗi (Error Handling): Không hiển thị chi tiết lỗi cơ sở dữ liệu hay stack trace ra ngoài giao diện người dùng. Ghi log lỗi vào file riêng.

## 2.6. Bảo mật Container với Dockerfile an toàn và Trivy

### 2.6.1. Bối cảnh và rủi ro từ container

Container (Docker) là công nghệ ảo hóa nhẹ, đóng gói ứng dụng cùng thư viện phụ thuộc. Tuy nhiên, việc xây dựng container không cẩn thận dẫn đến nhiều lỗ hổng: hình ảnh chứa các gói OS lỗi thời, thư viện có CVE, hard-code secret, chạy với quyền root, hoặc lớp cache giữ lại các thành phần cũ. Các rủi ro này liên quan trực tiếp đến A06:2021 – Vulnerable and Outdated Components và A08:2021 – Software and Data Integrity Failures trong OWASP Top 10.

### 2.6.2. Nguyên tắc viết Dockerfile an toàn

- Sử dụng base image nhẹ, được bảo trì tốt: Ưu tiên các image chính thức (alpine, slim) thay vì image full-featured, trừ khi thực sự cần thiết.
- Multi-stage build: Chỉ giữ lại các artifact cần thiết ở stage cuối, loại bỏ công cụ build, trình biên dịch, source code thừa.
- Chạy với non-root user: Tạo user riêng (ví dụ `umact_user`) có quyền hạn thấp, sử dụng chỉ thị USER. Tránh chạy ứng dụng dưới quyền root trong container.
- Không lưu secret trong Dockerfile: Không dùng ENV hay ARG để truyền trực tiếp mật khẩu, token. Sử dụng cơ chế quản lý secret từ orchestration (K8s Secrets, Docker secrets) hoặc file .env không có trong image.
- Quét các layer sau mỗi lần build: Sử dụng công cụ quét lỗ hổng (như Trivy) để phát hiện CVE và cấu hình sai trước khi push image lên registry.
- Cập nhật OS và thư viện: Chạy `apk update && apk upgrade` hoặc `apt-get update && apt-get upgrade` trong Dockerfile, đồng thời dùng cờ `--no-cache` để tránh dùng lại layer cũ chứa lỗ hổng.

### 2.6.3. Trivy – Công cụ quét bảo mật đa năng

- Trivy (do Aqua Security phát triển) là công cụ mã nguồn mở, chuyên phát hiện lỗ hổng và cấu hình sai trên container image, Infrastructure as Code (IaC), và hệ thống tệp. Trivy hoạt động theo cơ chế:

- Phân tích các lớp (layers) của Docker image để xác định danh sách gói phần mềm (package) trong hệ điều hành (Alpine, Debian, Ubuntu) và các thư viện ngôn ngữ (Python, Node.js, Java, Go, ...).
- Đối chiếu với cơ sở dữ liệu CVE (Common Vulnerabilities and Exposures) được cập nhật thường xuyên từ NVD, GitHub Security Advisories, v.v.
- Phát hiện cấu hình sai trong Dockerfile, Kubernetes YAML, Terraform, CloudFormation dựa trên các quy tắc bảo mật (ví dụ: container chạy privileged, mount sensitive path, chưa set non-root user).

#### **Ưu điểm nổi bật của Trivy:**

- Tốc độ quét nhanh (vài giây đến vài chục giây cho image cỡ vừa).
- Hỗ trợ nhiều hệ điều hành và kho ứng dụng.
- Dễ dàng tích hợp vào CI/CD (GitHub Actions, GitLab CI, Jenkins).
- Không yêu cầu kết nối Internet liên tục (có thể cache DB offline).

#### **Ví dụ lệnh quét một Docker image:**

```
bash
```

```
trivy image --severity HIGH,CRITICAL umact-api:latest
```

Kết quả quét sẽ liệt kê các lỗ hổng theo mức độ (LOW, MEDIUM, HIGH, CRITICAL), kèm mã CVE, phiên bản hiện tại và phiên bản đã được vá.

## **2.7. Nền tảng điều phối container cho môi trường hiện đại**

### **2.7.1. Khái niệm và vai trò**

Kubernetes (thường viết tắt là K8s) là một nền tảng mã nguồn mở dùng để tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng được đóng gói trong container. Ban đầu được Google phát triển dựa trên kinh nghiệm vận hành Borg và Omega, sau đó được chuyển giao cho Cloud Native Computing Foundation (CNCF). Kubernetes trở thành tiêu chuẩn thực tế cho orchestration container trong các kiến trúc microservices và cloud-native.

Trong bối cảnh bảo mật, Kubernetes vừa là giải pháp vừa là mục tiêu của các cuộc tấn công. Việc hiểu kiến trúc và các cơ chế bảo vệ của K8s là yêu cầu bắt buộc để xây dựng hạ tầng an toàn.

### **2.7.2. Kiến trúc tổng quan**

Một cluster Kubernetes gồm hai thành phần chính: **Control Plane(Master)** và **Worker Nodes**.

- Control Plane – Bộ não điều khiển
  - **API Server (kube-apiserver):** Cửa ngõ duy nhất để quản lý cluster (qua kubectl, REST API). Mọi thao tác đều qua API Server → đây là thành phần quan trọng nhất cần bảo vệ.

- **etcd:** Kho lưu trữ key-value phân tán, chứa toàn bộ cấu hình và trạng thái của cluster. Cần mã hóa và kiểm soát truy cập nghiêm ngặt.
  - **Scheduler:** Quyết định Pod chạy trên Node nào dựa trên tài nguyên và ràng buộc.
  - **Controller Manager:** Chạy các vòng lặp điều khiển (replication controller, deployment controller, v.v.) để duy trì trạng thái mong muốn.
- Worker Nodes – Nơi chạy ứng dụng
- **kubelet:** Agent chạy trên mỗi Node, đảm bảo container chạy đúng theo PodSpec.
  - **kube-proxy:** Quản lý các quy tắc mạng, chuyển tiếp lưu lượng đến Pod.
  - **Container Runtime:** Docker, containerd, CRI-O (giao tiếp với kubelet qua CRI).
- Các đối tượng cốt lõi (Resources)
- **Pod:** Đơn vị triển khai nhỏ nhất, gồm một hoặc nhiều container dùng chung network và storage.
  - **Deployment:** Quản lý bản sao (replica) của Pod, hỗ trợ rolling update và rollback.
  - **Service:** Cung cấp địa chỉ IP ổn định và cân bằng tải cho một nhóm Pod.
  - **Namespace:** Phân vùng ảo trong cluster để cách ly tài nguyên và quyền hạn.
  - **Secret:** Lưu trữ dữ liệu nhạy cảm (mật khẩu, token, khóa SSH) dưới dạng base64.
  - **NetworkPolicy:** Định nghĩa quy tắc tường lửa cho phép/ngăn chặn lưu lượng giữa các Pod hoặc từ bên ngoài.

### 2.7.3. Các rủi ro bảo mật đặc thù trong Kubernetes

Dựa trên tài liệu của OWASP Kubernetes Top 10 và CIS Benchmarks, các rủi ro phổ biến gồm:

| Rủi ro                       | Mô tả                                                                     | Hậu quả                                                      |
|------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------|
| API Server không được bảo vệ | API Server mở public không có xác thực hoặc TLS yếu                       | Kẻ tấn công gửi lệnh tùy ý, tạo Pod độc hại, đánh cắp Secret |
| RBAC cấu hình sai            | Gán quyền quá lớn cho Service Account hoặc người dùng (vd: cluster-admin) | Leo thang đặc quyền, chiếm toàn bộ cluster                   |

|                                    |                                                                      |                                                                                |
|------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Secret ở dạng plaintext            | Secret không được mã hóa trong etcd, hoặc lưu trong file .env, image | Rò rỉ thông tin nhạy cảm                                                       |
| Pod Security không được áp dụng    | Pod chạy ở chế độ privileged, mount Docker socket, hostPID           | Thoát container ra Node, kiểm soát host                                        |
| NetworkPolicy không được thiết lập | Mặc định K8s không chặn lưu lượng giữa các Pod                       | Kẻ tấn công sau khi xâm nhập một Pod có thể di chuyển ngang (lateral movement) |
| Hình ảnh container có lỗ hổng      | Pod dùng image chứa CVE hoặc phần mềm cũ                             | Bị khai thác từ bên trong                                                      |
| Cập nhật và vá lỗi chậm            | CVE trong kubelet, kube-proxy, hoặc container runtime không được vá  | Tấn công từ xa vào Node                                                        |

## 2.8. Kiến trúc và bảo mật trong môi trường hiện đại

### 2.8.1. Khái niệm Cloud Native

Cloud Native là một cách tiếp cận để xây dựng và vận hành các ứng dụng tận dụng tối đa những lợi ích của mô hình điện toán đám mây. Theo định nghĩa của Cloud Native Computing Foundation (CNCF), công nghệ cloud native cho phép các tổ chức xây dựng và chạy các ứng dụng có khả năng mở rộng (scalable) trong các môi trường năng động như đám mây công cộng, riêng tư hoặc lai. Điểm cốt lõi là sử dụng **container**, **orchestration** (Kubernetes), **microservices**, **declarative APIs** (khai báo trạng thái mong muốn) và **DevOps** (tích hợp phát triển với vận hành).

Trái với các ứng dụng truyền thống (monolithic, triển khai trên máy chủ vật lý hoặc VM với chu kỳ phát hành dài), ứng dụng cloud native được thiết kế để **nhANH NHẠY, ĐÀN HỒI và có khả năng phục hồi cao**. Bảo mật trong cloud native không thể là một lớp phủ thêm vào cuối, mà phải được **tích hợp ngay từ đầu** (Shift Left) và xuyên suốt vòng đời.

### 2.8.2. Các đặc trưng chính của Cloud Native

- **Đóng gói bằng container:** Ứng dụng và các phụ thuộc được đóng gói vào container độc lập với hạ tầng. Container nhẹ, di động, cho phép triển khai nhất quán giữa các môi trường (dev, test, production).
- **Quản lý vòng đời tự động (Orchestration):** Kubernetes trở thành hệ điều hành của cloud native, tự động hóa việc triển khai, mở rộng, cập nhật và phục hồi ứng dụng.

- **Kiến trúc microservices:** Ứng dụng được phân rã thành các dịch vụ nhỏ, độc lập, mỗi dịch vụ có thể được phát triển, triển khai và mở rộng riêng rẽ. Điều này làm tăng bề mặt tấn công (nhiều API hơn) nhưng cũng cho phép cô lập lỗ hổng tốt hơn.
- **Khai báo (declarative) và Infrastructure as Code (IaC):** Trạng thái mong muốn của hệ thống (Pod, Service, Ingress, NetworkPolicy, v.v.) được mô tả bằng các file cấu hình (YAML) và lưu trữ trong Git. Hạ tầng được quản lý như code, cho phép kiểm tra, review và tự động hóa.
- **DevOps và CI/CD Pipeline:** Tự động hóa từ build, test, bảo mật đến triển khai. Bảo mật được tích hợp vào pipeline (DevSecOps) thay vì là bước kiểm tra riêng cuối cùng.
- **Đàn hồi và khả năng chịu lỗi:** Ứng dụng được thiết kế để xử lý lỗi của từng thành phần (failure-oriented design). Các cơ chế như health checks, retry, circuit breaker được sử dụng phổ biến.



## CHƯƠNG 3. THỰC NGHIỆM / GIẢI PHÁP ĐỀ XUẤT

Để đảm bảo tính khách quan và sát với thực tế doanh nghiệp, nhóm tiến hành thực nghiệm trực tiếp trên dự án thương mại điện tử UmaCT do nhóm tự phát triển gần đây.

### 3.1. Môi trường và công cụ thử nghiệm

- **Hệ điều hành / máy ảo:** Môi trường Localhost chạy trên hệ điều hành Windows, kết hợp với Docker Desktop để giả lập cụm máy chủ Kubernetes nội bộ (K8s)
- **Công cụ sử dụng:**
  - Công cụ kiểm thử tĩnh (SAST): Trợ lý AI Antigravity Agent.
  - Công cụ kiểm thử động (DAST): OWASP ZAP (phiên bản 2.17.0).
  - Công cụ DevSecOps: Docker, nền tảng quét lỗ hổng Trivy.
  - Công cụ quản trị hạ tầng: Kubernetes (kubectl).
- **Mục tiêu thử nghiệm (lab):** Hệ thống UmaCT E-Commerce bao gồm Frontend (PHP) và Backend API (FastAPI - Python).

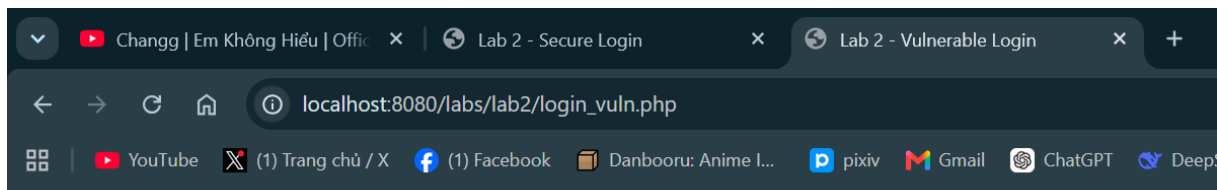
### 3.2. Các bước thực hiện

1. Bước 1: Phân tích mã nguồn tĩnh (SAST) và Khai thác SQL Injection
  - Sử dụng Antigravity Agent quét toàn bộ thư mục mã nguồn PHP và Python để tìm các lỗ hổng cấu hình và logic.
  - Dụng kịch bản tấn công SQL Injection trên file login\_vuln.php (sử dụng truy vấn nối chuỗi thô sơ). Kê tấn công nhập payload ' OR 1=1 -- vào trường username để vượt qua khâu xác thực mà không cần mật khẩu.
2. Bước 2: Kiểm thử động ứng dụng (DAST) bằng OWASP ZAP
  - Khởi chạy Backend FastAPI tại cổng 8000 và Frontend tại cổng 8080.
  - Cấu hình OWASP ZAP làm proxy trung gian. Sử dụng tính năng Active Scan và AJAX Spider rà quét toàn bộ các endpoint API (như /api/favorites/toggle và /docs) nhằm phát hiện các điểm yếu về cấu hình Header và rò rỉ thông tin.
3. Bước 3: Đóng gói vùng chứa (Container Security) và Quét bằng Trivy
  - Xây dựng Dockerfile áp dụng kỹ thuật Multi-stage build và thiết lập base image an toàn python:3.10-alpine.
  - Tạo người dùng đặc quyền thấp (non-root) umact\_user để chạy tiến trình.
  - Sử dụng công cụ Trivy quét image umact-api:secure vừa khởi tạo để đánh giá lỗ hổng hệ điều hành (OS) và thư viện phụ thuộc (Dependencies).
4. Bước 4: Thiết lập bảo mật hạ tầng Kubernetes
  - Tạo file umact-pod-security.yaml cấu hình securityContext để cấm leo thang đặc quyền (allowPrivilegeEscalation: false).
  - Tạo file umact-network-policy.yaml thiết lập hàng rào mạng ảo, cô lập Backend. Chạy lệnh kubectl apply để áp dụng chính sách và kubectl describe để kiểm tra luồng mạng.

### 3.3. Kết quả và nhận xét

Kết quả kiểm thử mã nguồn và ứng dụng (SAST/DAST):

- **Về SAST:** Báo cáo ghi nhận 15 lỗi hỏng nghiêm trọng. Trong đó, hệ thống mắc lỗi **A01:2021 (Broken Access Control)** khi toàn bộ các Admin API không có middleware xác thực; và lỗi **A02:2021 (Cryptographic Failures)** do lưu mật khẩu dạng plaintext. Việc khai thác SQL Injection thành công bằng payload ' OR 1=1 -- đã chứng minh sự yếu kém của việc không lọc dữ liệu đầu vào.
- **Về DAST:** OWASP ZAP phát hiện ứng dụng thiếu hàng loạt Security Headers (CSP, X-Frame-Options) làm tăng nguy cơ Clickjacking. Đặc biệt, lỗi **Application Error Disclosure** bị phát hiện khi server trả về HTTP 500 lộ thông tin debug hệ thống.



#### Vulnerable Login (SQLi Demonstration)

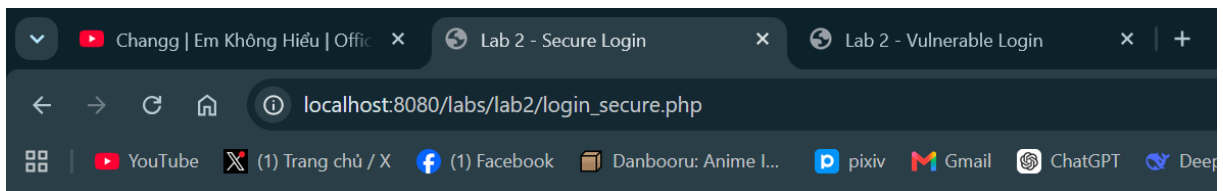
Login successful. Welcome, admin!

Username:

Password:

Login

*Hình 3.1. Khai thác thành công lỗi SQL Injection trên mã nguồn yếu kém  
(Nguồn: Nhóm thực hiện)*



## Secure Login (Prepared Statements)

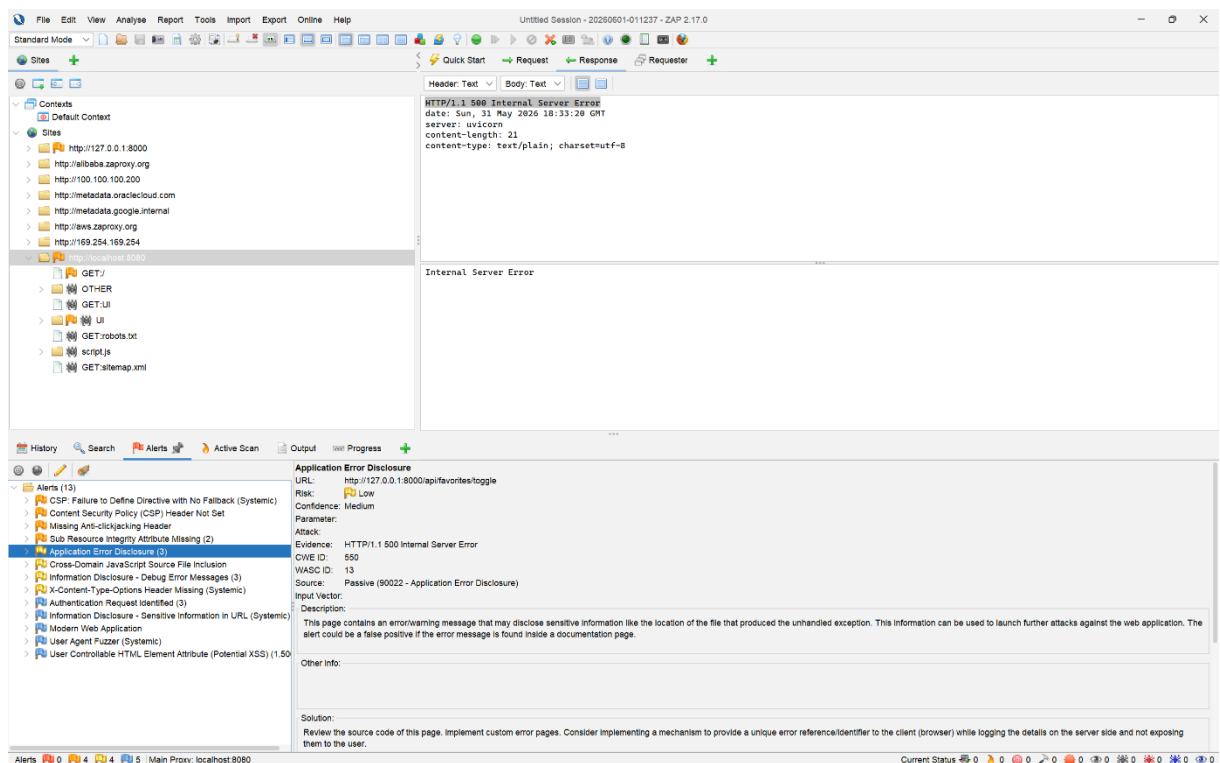
Invalid credentials.

Username:

Password:

Login

Hình 3.2. Chặn đứng SQL Injection sau khi áp dụng Prepared Statements  
(Nguồn: Nhóm thực hiện)



Hình 3.3. Lỗi Application Error Disclosure phát hiện bởi OWASP ZAP (Nguồn: Nhóm tự thực hiện)

## Kết quả rà soát Container (DevSecOps):

Ban đầu, Trivy phát hiện 1 lỗ hổng hệ điều hành (CVE-2026-34743) và 14 lỗ hổng thư viện Python (HIGH/MEDIUM). Nhóm đã khắc phục bằng cách nâng cấp OS

và các công cụ lỗi. Tuy nhiên, rủi ro về "Thư viện nhúng ngầm" (Vendored Dependencies) xuất hiện khi gói jaraco.context lỗi thời bị giấu bên trong setuptools. Sau khi nỗ lực tận gốc, hệ thống đã đạt trạng thái Zero-Vulnerability (Total: 0).

| umact-api:secure (alpine 3.23.4)                                |                |          |        |                   |               |                                                                                                                                                                                                       |
|-----------------------------------------------------------------|----------------|----------|--------|-------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| =====                                                           |                |          |        |                   |               |                                                                                                                                                                                                       |
| Total: 1 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 0, CRITICAL: 0)  |                |          |        |                   |               |                                                                                                                                                                                                       |
| Library                                                         | Vulnerability  | Severity | Status | Installed Version | Fixed Version | Title                                                                                                                                                                                                 |
| xz-libs                                                         | CVE-2026-34743 | MEDIUM   | fixed  | 5.8.2-r0          | 5.8.3-r0      | xz: XZ Utils: Denial of Service via buffer overflow in index decoding...<br><a href="https://avd.aquasec.com/nvd/cve-2026-34743">https://avd.aquasec.com/nvd/cve-2026-34743</a>                       |
| Total: 14 (UNKNOWN: 0, LOW: 1, MEDIUM: 6, HIGH: 7, CRITICAL: 0) |                |          |        |                   |               |                                                                                                                                                                                                       |
| Library                                                         | Vulnerability  | Severity | Status | Installed Version | Fixed Version | Title                                                                                                                                                                                                 |
| jaraco.context (METADATA)                                       | CVE-2026-23949 | HIGH     | fixed  | 5.3.0             | 6.1.0         | jaraco.context: jaraco.context: Path traversal via malicious tar archives<br><a href="https://avd.aquasec.com/nvd/cve-2026-23949">https://avd.aquasec.com/nvd/cve-2026-23949</a>                      |
| pip (METADATA)                                                  | CVE-2023-5752  | MEDIUM   |        | 23.0.1            | 23.3          | pip: Mercurial configuration injectable in repo revision when installing via pip<br><a href="https://avd.aquasec.com/nvd/cve-2023-5752">https://avd.aquasec.com/nvd/cve-2023-5752</a>                 |
|                                                                 | CVE-2025-8869  |          |        |                   | 25.3          | pip: pip missing checks on symbolic link extraction<br><a href="https://avd.aquasec.com/nvd/cve-2025-8869">https://avd.aquasec.com/nvd/cve-2025-8869</a>                                              |
|                                                                 | CVE-2026-3219  |          |        |                   | 26.1          | pip: pip: Incorrect file installation due to improper archive handling<br><a href="https://avd.aquasec.com/nvd/cve-2026-3219">https://avd.aquasec.com/nvd/cve-2026-3219</a>                           |
|                                                                 | CVE-2026-6357  |          |        |                   |               | pip: pip: Arbitrary code execution or information disclosure via malicious wheel package...<br><a href="https://avd.aquasec.com/nvd/cve-2026-6357">https://avd.aquasec.com/nvd/cve-2026-6357</a>      |
|                                                                 | CVE-2026-1703  | LOW      |        |                   | 26.0          | pip: pip: Information disclosure via path traversal when installing crafted wheel archives...<br><a href="https://avd.aquasec.com/nvd/cve-2026-1703">https://avd.aquasec.com/nvd/cve-2026-1703</a>    |
| python-multipart (METADATA)                                     | CVE-2024-53981 | HIGH     |        | 0.0.9             | 0.0.18        | python-multipart: python-multipart has a DoS via deformation 'multipart/form-data' boundary<br><a href="https://avd.aquasec.com/nvd/cve-2024-53981">https://avd.aquasec.com/nvd/cve-2024-53981</a>    |
|                                                                 | CVE-2026-24486 |          |        |                   | 0.0.22        | python-multipart: Python-Multipart: Arbitrary file write via path traversal vulnerability<br><a href="https://avd.aquasec.com/nvd/cve-2026-24486">https://avd.aquasec.com/nvd/cve-2026-24486</a>      |
|                                                                 | CVE-2026-42561 |          |        |                   | 0.0.27        | Python-Multipart is a streaming multipart parser for Python. Prior to .....<br><a href="https://avd.aquasec.com/nvd/cve-2026-42561">https://avd.aquasec.com/nvd/cve-2026-42561</a>                    |
|                                                                 | CVE-2026-40347 | MEDIUM   |        |                   | 0.0.26        | python-multipart: Python-Multipart: Denial of Service via crafted multipart/form-data requests<br><a href="https://avd.aquasec.com/nvd/cve-2026-40347">https://avd.aquasec.com/nvd/cve-2026-40347</a> |
| starlette (METADATA)                                            | CVE-2024-47874 | HIGH     |        | 0.38.6            | 0.40.0        | starlette: Starlette Denial of service (DoS) via multipart/form-data<br><a href="https://avd.aquasec.com/nvd/cve-2024-47874">https://avd.aquasec.com/nvd/cve-2024-47874</a>                           |
|                                                                 | CVE-2025-54121 | MEDIUM   |        |                   | 0.47.2        | starlette: Starlette denial-of-service<br><a href="https://avd.aquasec.com/nvd/cve-2025-54121">https://avd.aquasec.com/nvd/cve-2025-54121</a>                                                         |
| wheel (METADATA)                                                | CVE-2026-24049 | HIGH     |        | 0.45.1            | 0.46.2        | wheel: wheel: Privilege Escalation or Arbitrary Code Execution via malicious wheel file...<br><a href="https://avd.aquasec.com/nvd/cve-2026-24049">https://avd.aquasec.com/nvd/cve-2026-24049</a>     |

Hình 3.4. Kết quả quét Trivy khi chưa vá lỗi (Nguồn: Nhóm tự thực hiện)

|                                                                                                       |            |   |   |
|-------------------------------------------------------------------------------------------------------|------------|---|---|
| usr/local/lib/python3.10/site-packages/requests-2.34.2.dist-info/METADATA                             | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools-82.0.1.dist-info/METADATA                           | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/autocommand-2.2.2.dist-info/METADATA        | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/backports.tarfile-1.2.0.dist-info/METADATA  | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/importlib_metadata-8.7.1.dist-info/METADATA | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/jaraco.text-4.0.0.dist-info/METADATA        | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/jaraco.context-6.1.0.dist-info/METADATA     | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/jaraco.functors-4.4.0.dist-info/METADATA    | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/more_itertools-10.8.0.dist-info/METADATA    | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/packaging-26.0.dist-info/METADATA           | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/platformdirs-4.4.0.dist-info/METADATA       | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/tomli-2.4.0.dist-info/METADATA              | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/wheel-0.46.3.dist-info/METADATA             | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/setuptools/_vendor/zip-3.23.0.dist-info/METADATA               | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/six-1.17.0.dist-info/METADATA                                  | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/starlette-1.2.1.dist-info/METADATA                             | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/typing_extensions-4.15.0.dist-info/METADATA                    | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/typing_inspection-0.4.2.dist-info/METADATA                     | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/urllib3-2.7.0.dist-info/METADATA                               | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/uvicorn-0.30.6.dist-info/METADATA                              | python-pkg | 0 | - |
| usr/local/lib/python3.10/site-packages/wheel-0.47.0.dist-info/METADATA                                | python-pkg | 0 | - |

Legend:  
- '-': Not scanned  
- '0': Clean (no security findings detected)

*Hình 3.5. Kết quả quét Trivy an toàn tuyệt đối sau khi vá lỗi*

*(Nguồn: Nhóm tự thực hiện)*

### **Kết quả cấu hình Kubernetes:**

Lệnh `kubectl describe networkpolicy` xác nhận chính sách Ingress đã hoạt động. Backend bị chặn hoàn toàn khỏi Internet và chỉ chấp nhận kết nối từ các Pod có nhãn `app: umact-frontend`.

```

C:\Windows\System32>cd D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4
D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4>kubectl apply -f umact-pod-security.yaml
deployment.apps/umact-backend unchanged
D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4>kubectl apply -f umact-pod-security.yaml
deployment.apps/umact-backend unchanged
D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4>kubectl describe networkpolicy block-external-access
Name:          block-external-access
Namespace:     default
Created on:    2026-06-02 21:53:39 +0700 +07
Labels:       <none>
Annotations:  <none>
Spec:
  PodSelector:  app=umact-backend
  Allowing ingress traffic:
    To Port: 8000/TCP
    From:
      PodSelector: app=umact-frontend
  Not affecting egress traffic
  Policy Types: Ingress
D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4>
D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4>kubectl get pods -l app=umact-backend -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE             NOMINATED NODE   READINESS GATES
umact-backend-7c7f85f7d9-1bfwm      1/1     Running   1 (73s ago)  115m  10.244.0.4    desktop-control-plane   <none>            <none>
D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4>kubectl get pod -l app=umact-backend -o jsonpath='{.items[0].spec.securityContext}'
{"runAsNonRoot":true,"runAsUser":1001}
D:\K23CNT3-LucVanSon-2310900087\K23CNT3_LucVanSon_2310900087_Project4>

```

Hình 3.6. Kiểm tra luồng mạng bằng Network Policy (Nguồn: Nhóm tự thực hiện)

### 3.4. Đề xuất biện pháp phòng chống / khắc phục

Từ các kết quả thực nghiệm, nhóm đề xuất các biện pháp phòng thủ chiều sâu như sau:

- **Tầng Mã nguồn (AppSec):** Thay thế toàn bộ các truy vấn SQL nối chuỗi bằng kỹ thuật Prepared Statement (PDO) để tách bạch dữ liệu và câu lệnh. Triển khai thuật toán băm bcrypt cho mật khẩu và di dời toàn bộ API Keys (Hardcoded Credentials) vào file môi trường (.env).
- **Tầng Cấu hình Ứng dụng:** Thiết lập các trường bảo mật HTTP Header tiêu chuẩn (X-Content-Type-Options: nosniff, X-Frame-Options: DENY). Chuyển ứng dụng sang môi trường Production để tắt chế độ Debug, cấu hình lại hệ thống trả về mã lỗi chung (ví dụ: HTTP 400 hoặc 404) thay vì lộ stack-trace HTTP 500.
- **Tầng Quản lý Vùng chứa (Container):** Khi triển khai CI/CD Pipeline, cần bắt buộc thêm cờ --no-cache trong quá trình build các bản vá bảo mật để tránh việc Docker tái sử dụng các layer cache chứa thư viện cũ. Thường xuyên rà quét thành phần phần mềm (SCA) để ngăn ngừa rủi ro từ Thư viện nhúng ngầm (Vendored Dependencies).
- **Tầng Hạ tầng (Kubernetes):** Mặc dù NetworkPolicy đã thiết lập bảo mật chiều vào (Ingress), ở môi trường thực tế, hệ thống cần bổ sung Default Deny cho cả chiều ra (Egress). Điều này giúp triệt tiêu hoàn toàn rủi ro hacker dùng Backend làm bàn đạp thực hiện tấn công Server-Side Request Forgery (SSRF) ra mạng bên ngoài. Bổ sung cờ readOnlyRootFilesystem: true để khóa chết hệ thống tệp tin của Pod.

# KẾT LUẬN

## Kết quả đạt được

Trong thời gian thực hiện tiểu luận, nhóm đã hoàn thành mục tiêu nghiên cứu và triển khai thành công quy trình kiểm thử, bảo mật toàn diện cho hệ thống UmaCT E-Commerce. Cụ thể, nhóm đã đạt được các kết quả nổi bật sau:

- **Về kiểm thử ứng dụng (AppSec):** Áp dụng phân tích mã nguồn tĩnh (SAST) để phát hiện và vá lỗi SQL Injection thông qua kỹ thuật Prepared Statement. Sử dụng OWASP ZAP kiểm thử động (DAST), phát hiện các lỗ hổng cấu hình Header và rò rỉ lỗi ứng dụng.
- **Về bảo mật vùng chứa (Container Security):** Triển khai quy trình DevSecOps, xây dựng Dockerfile an toàn theo nguyên lý đặc quyền tối thiểu. Sử dụng Trivy quét và vá 100% các lỗ hổng hệ điều hành và thư viện phụ thuộc, đưa image về trạng thái Zero-Vulnerability.
- **Về bảo mật hạ tầng Cloud-native:** Xây dựng kịch bản Kubernetes Manifest áp dụng PodSecurity và NetworkPolicy để thiết lập mô hình Zero-Trust, cô lập hoàn toàn API backend khỏi các truy cập trái phép.

## Hạn chế và hướng phát triển

- **Hạn chế:** Hệ thống kiểm thử mới chỉ được mô phỏng trên môi trường Localhost thay vì môi trường Cloud phân tán thực tế. Việc tích hợp công cụ kiểm thử tự động chưa được cấu hình hoàn chỉnh bằng CI/CD tools như GitHub Actions.
- **Hướng phát triển:** Trong tương lai, nhóm dự định triển khai các CNI Plugin thế hệ mới như Cilium (sử dụng công nghệ eBPF) để tối ưu hóa hiệu năng kiểm tra NetworkPolicy. Đồng thời, tích hợp SAST/SCA tự động vào GitHub Actions pipeline.

## TÀI LIỆU THAM KHẢO

- [1] NIST, *Security Considerations in the Information System Development Life Cycle*, NIST Special Publication 800-64 Rev. 1, National Institute of Standards and Technology, 2004.
- [2] OWASP Foundation, *OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks*, OWASP, 2021.
- [3] R. Kissel, K. Stine, M. Scholl, H. Rossman, J. Fahlsing, *Security Considerations in the System Development Life Cycle*, NIST Special Publication 800-64 Rev. 2, National Institute of Standards and Technology, 2008.
- [4] OWASP ZAP Project, *OWASP Zed Attack Proxy (ZAP) – Official Documentation*, OWASP Foundation, 2025.
- [5] PortSwigger, *Burp Suite Community Edition – Release Notes and Features*, PortSwigger Ltd, 2025–2026.
- [6] DVWA Project, *Damn Vulnerable Web Application (DVWA) – Official Documentation*.
- [7] Trivy Project, *Trivy – Comprehensive Vulnerability Scanner for Containers, IaC, and Artifacts*.
- [8] OWASP Foundation, *OWASP API Security Top 10 – 2023*, OWASP, 2023.
- [9] OWASP Foundation, *OWASP Cheat Sheet Series (OCSS)*, OWASP.
- [10] NIST, *General Access Control Guidance for Cloud Systems*, NIST Special Publication 800-210, National Institute of Standards and Technology, 2020.
- [11] Kubernetes Authors, *Kubernetes Documentation: Pod Security Standards (PSS) and Network Policies*, Kubernetes.
- [12] NIST, *Implementation of DevSecOps for a Microservices-based Application with Service Mesh*, NIST Special Publication 800-204C, National Institute of Standards and Technology, 2022.
- [13] PortSwigger, *\*Burp Suite Professional / Community 2025.12.3 – Release Notes\**, PortSwigger Ltd, 2026.
- [14] Nguyễn Tài Tuyên, *Bài giảng An toàn và bảo mật hệ thống thông tin - Bài 7: Bảo mật ứng dụng và phần mềm*, Đại học Nguyễn Trãi, 2026.
- [15] Secure SDLC (SSDLC)? (n.d.). <https://vietnamese.opswat.com/blog/what-is-secure-sdlc>
- [16] Tổng quan về Các công cụ SAST trong AppSec 2025. (n.d.). <https://cyberjutsu.io/blog/tong-quan-ve-cac-cong-cu-sast-trong-appsec-2025>
- [17] OWASP là gì? OWASP Top 10 là gì? Cách loại bỏ lỗ hổng ... (n.d.). <https://vietnix.vn/owasp-la-gi/>



## PHỤ LỤC

### PHỤ LỤC A: ĐÁNH GIÁ VÀ XỬ LÝ LỖ HỔNG ỨNG DỤNG (WEB VULNERABILITY)

#### 1. Phân tích lỗ hổng SQL Injection

- Trong quá trình thực nghiệm tại dự án UmaCT, nhóm đã thực hiện đánh giá lỗ hổng SQL Injection trên các biểu mẫu đăng nhập. Vấn đề cốt lõi của các đoạn mã có lỗ hổng là việc cho phép dữ liệu đầu vào từ người dùng (như tên đăng nhập, mật khẩu) được nối trực tiếp vào câu lệnh truy vấn cơ sở dữ liệu mà không thông qua cơ chế kiểm duyệt. Điều này tạo cơ hội cho kẻ tấn công chèn các lệnh SQL giả mạo để bypass xác thực.

#### 2. Chiến lược khắc phục (Mitigation)

- Để xử lý vấn đề trên, nhóm đã áp dụng kỹ thuật "Truy vấn tham số hóa" (Parameterized Queries). Thay vì nối chuỗi, hệ thống tách biệt phần lệnh thực thi và phần dữ liệu đầu vào.
- **Nguyên tắc:** Dữ liệu người dùng được xử lý như một giá trị thuần túy, không bao giờ được trình thông dịch SQL coi là lệnh thực thi.
- **Kết quả:** Kẻ tấn công không thể thay đổi logic của câu lệnh truy vấn bằng các ký tự đặc biệt, từ đó triệt tiêu hoàn toàn khả năng khai thác lỗ hổng Injection.

### PHỤ LỤC B: CHIẾN LƯỢC TỐI ƯU HÓA VÀ BẢO MẬT CONTAINER (DOCKER)

#### 1. Mô hình xây dựng Multi-stage

- Nhóm đã áp dụng kiến trúc "Multi-stage Build" để tối ưu hóa môi trường runtime.
- **Giai đoạn biên dịch (Build stage):** Chứa các thư viện, công cụ biên dịch cần thiết nhưng có kích thước lớn và chứa nhiều lỗ hổng tiềm ẩn.
- **Giai đoạn vận hành (Runtime stage):** Chỉ bao gồm các tệp thực thi cần thiết và mã nguồn ứng dụng, giúp giảm thiểu bề mặt tấn công (attack surface) bằng cách loại bỏ hoàn toàn các trình biên dịch không cần thiết.

#### 2. Nguyên tắc đặc quyền tối thiểu (Least Privilege)

- Mọi container trong dự án đều được cấu hình để chạy dưới quyền của một người dùng thông thường (non-root user). Việc này ngăn chặn nguy cơ "Container Escape", đảm bảo rằng nếu ứng dụng bị xâm nhập, kẻ tấn công cũng không thể chiếm quyền điều khiển trên máy chủ vật lý (host) hoặc các tiến trình hệ thống quan trọng.

## PHỤ LỤC C: BẢO MẬT HẠ TẦNG ĐIỀU PHỐI (KUBERNETES)

### 1. Cấu hình bảo mật Pod (Security Context)

- Nhóm triển khai các chính sách bảo mật tầng Pod để siết chặt quyền hạn:
- **ReadOnlyRootFilesystem:** Cấu hình hệ thống tệp chỉ đọc đối với vùng lưu trữ ứng dụng, ngăn chặn kẻ tấn công cài đặt mã độc hoặc thay đổi tệp tin hệ thống sau khi đã xâm nhập.
- **Chặn leo thang đặc quyền (AllowPrivilegeEscalation):** Cấm tiến trình con có quyền hạn cao hơn tiến trình cha.

### 2. Chính sách mạng (Network Policy) - Mô hình Zero-Trust

- Thay vì để các dịch vụ trong cụm tự do giao tiếp, chính sách Network Policy được thiết lập để:
- Cô lập hoàn toàn ứng dụng khỏi các lưu lượng truy cập trái phép.
- Chỉ cho phép duy nhất các dịch vụ Frontend kết nối tới Backend thông qua các cổng đã được định nghĩa.
- Mọi yêu cầu truy cập từ các nguồn khác đều bị chặn mặc định theo mô hình tin tưởng bằng không (Zero-Trust).